SOLID-C

Компилятор языка Си в системе ESTEX

ВВЕДЕНИЕ

Одной из целей разработки оптимизирующего компилятора SOLID C было создание инструментального средства, которое позволяет писать и саму операционную систему, и программное обеспечение для нее. Для достижения этой цели в кодогенератор введены различные дополнительные средства. Так, разработаны средства автоматического распределения регистров, благодаря которым удалось существенно повысить эффективность объектного кода. В результате стало возможным писать на высоко-уровневом языке (которым является Си) даже системы, работающие в условиях жестких ограничений по памяти и реальному масштабу времени. При этом почти полностью устранена необходимость программирования на Ассемблере.

Основная цель данного руководства - дать пользователям представление о применении компилятора в рамках операционной системы ESTEX и некоторых особенностях этого компилятора.

SOLID C является двухпроходным компилятором языка, преобразующим исходный си-текст в программу на языке Ассемблера. Компилятор состоит из двух файлов "cc1.exe" и "cc2.exe", выполняющих соответственно первый и второй проходы.

На первом проходе входной си-текст преобразуется в объектный код (т-код), сохраняемый в файле с расширением ".tmc" (по-умолчанию). На первом проходе производятся также диагностика ошибок и выдача предупреждающих сообщений.

На втором проходе полученный т-код преобразуется в ассемблерный листинг, сохраняемый в файле с расширением ".asm" (по-умолчанию).

Следует отметить тот факт, что поскольку данный компилятор является оптимизирующим, то время, затраченное на компиляцию си-программы возрастает от 2-х и более раз, по сравнению с обычными (не оптимизирующими) си-компиляторами.

ГЕНЕРАТОР ОБЪЕКТНОГО КОДА

Этот компонент (cc1.exe) компилятора обеспечивает первый проход. Он производит разбор исходного си-текста, его контроль на допустимость синтаксиса и семантики. В результате первого прохода появляется промежуточный файл с объектным кодом, т.н. Т-код.

Стандартным расширением имен файлов, содержащих т-код, является ".tmc".

Команда запуска компилятора первого прохода имеет следующий формат:

```
CC1 [опция] имя_файла
```

Имя файла может содержать букву диска и путь, при этом максимальная длина строки не может превышать 80 символов. Обычно для исходных си-файлов используется расширение ".c". Если расширение входного файла ".c", то его можно не указывать.

Опции генератора объектного кода

-c	В отличие от стандартного языка Си, компилятор SOLID C обрабатывает гнездование (вложение) комментариев. Данная опция отключает режим гнездования комментариев.
-dNAME	Определить макро-определение с именем "NAME". Между буквой опции и именем "NAME" пробелы не допускаются.
-e	Режим переопределения выдачи сообщений в файл ошибок, имеющего расширение ".err". Файл создается всегда по текущему пути исходного ".c" файла.
-jN	Установка максимального числа ошибок компиляции. Происходит прерывание компиляции, если число ошибок равно или превышает "N". Между буквой опции и числом "N" пробелы не допускаются. Например: -j30 (прерывание процесса компиляции при 30 ошибках).
-k	Режим совместимости со стандартным языком Си (стандарт K&R). Разрешает делать неявные объявления функций и параметров. Данную опцию необходимо использовать при компиляции "чужых" исходников.
-m	Режим вывода на экран статистики по таблицам компилятора.
-oNAME	Задать имя для выходного файла с т-кодом. Максимальная длина строки полного пути (диск\каталог\имя) файла составляет 80 символов. Между буквой опции и именем "NAME" пробелы не допускаются.
-rP:S:H	Установка отношения размеров частей рабочей области компилятора: P - пула, S - таблицы символов, H - хеш-таблицы. По-умолчанию используется соотношение 13:6:4. В случае переполнения какой-либо части, используйте эту опцию для назначения другого соотношения частей рабочей области. Между буквой опции и строкой "P:S:H" пробелы не допускаются.
-x	Режим строгой совместимости с MSX-C компилятором. При этом расширение файла т-кода меняется с ".tmc" на ".tco".

КОДОГЕНЕРАТОР

Кодогенератор (cc2.exe) выполняет второй проход компилятора. Он читает файл с т-кодом и создает выходной файл на языке Ассемблера.

Команда запуска компилятора второго прохода имеет следующий формат:

CC2 [опция] имя_файла

Если у входного файла с т-кодом расширение не указано, то используется стандартное расширение ".tmc". Если не указано расширение для выходного файла — используется расширение ".asm". Максимальная длина строки "диск\каталог\имя_файла" должна быть не более 64 символов.

Если путь расположения для выходного файла не задан, то по-умолчанию он создается по текущему пути расположения входного файла т-кода.

Опции кодогенератора

-k	Не удалять входной файл т-кода (.tmc) по завершению работы кодогенератора.
-oNAME	Задать имя для выходного asm-файла. Максимальная длина строки полного пути (диск\каталог\имя) файла составляет 64 символа. Между буквой опции и именем "NAME" пробелы не допускаются.
-rN	Резервирование области под таблицу символов компилятора размером N байтов (N - десятичный формат). Между буквой опции и числом "N" пробелы не допускаются. Например: -r2000 (резервируется таблица символов размером в 2000 байтов).
-uX	Определить пост-символ "X" для имен в asm-листинге. По-умолчанию используется символ "_". Например: -u@ (использовать пост-символ "@" для имен в выходном asm-листинге).
-q	"Тихий" режим. Индикаторы процесса компиляции "---" и "+++" на экран не выводятся.

ОСОБЕННОСТИ КОМПИЛЯТОРА SOLID C ДЛЯ ОС ESTEX

Как было сказано выше, целью разработки данного Си-компилятора было создание продукта, с помощью которого можно было бы (пользуясь всеми преимуществами языка высокого уровня) разрабатывать программы, удовлетворяющие жестким требованиям реального масштаба времени и ограничений на память. В данной главе описаны особенности компилятора с этой точки зрения. Кроме того, здесь будут даны рекомендации по программированию на SOLID C.

Компилятор SOLID C может работать в двух различных режимах. Один из них является "собственным" для данного компилятора, а второй - служит для обеспечения совместимости со стандартом языка Си. В этом режиме все арифметические преобразования выполняются точно так же, как в стандартном языке Си. Самое большое различие между этим режимом и "собственным" состоит в том, что в последнем не выполняются автоматические преобразования форматов при переходах от типа "char" к типу "int". По-умолчанию, компилятор работает в "собственном" режиме. Для смены режима работы компилятора служит опция "-k" на первом проходе компилятора.

Собственный режим компилятора

Для данного режима характерны два новых формата: целой константы (intconst) и символьной константы (charconst). Обычно указанные форматы используются идентично с соответствующими форматами "int" и "char", однако в случаях анализа типов операндов и выражений, и приведения их к соответствию, указанные два формата рассматриваются как различные.

Как упоминалось выше, в этом режиме не производится автоматическое преобразование форматов при переходе от типа "char" к "int". Действуют следующие соглашения о преобразованиях типов:

unsigned	+ ? →	unsigned
int	+ ? →	int
char	+ ? →	char
intconst	+ ? →	intconst
charconst	+ ? →	charconst

Режим совместимости со стандартом языка Си

Выполняются все арифметические преобразования, действительные в рамках реализации стандартного языка Си. Единственное различие состоит в том, что компилятор SOLID C интерпретирует тип "char", как значение без знака.

Для этого режима действуют следующие правила преобразования типов:

char	→	int
unsigned	+ ? →	unsigned
int	+ int →	int

Рекурсивные функции

Как известно, все функции стандартного языка Си рекурсивные. В среде компилятора SOLID C можно писать также и не рекурсивные функции. Для различия функций этих двух типов введены и зарезервированы следующие ключевые слова: "recursive" – рекурсивная и "nonrec" – не рекурсивная. Эти ключевые слова записываются в заголовке функции перед ее именем и служат описателями режимов ее выполнения. Это показано в следующем примере:

```
nonrec main()
{
   printf("Hello, world!\n");
}

recursive
int factorial(n)
int n;
{
   return(n > 1) ? n*factorial(n-1) : 1;
}

int max(a,b)
int a,b
{
   return(a > b) ? a : b;
}
```

В приведенном примере даны определения: не рекурсивной функции main() и рекурсивной функции factorial(). Для функции max() описатель режима выполнения функции в явном виде не задан, поэтому она будет выполняться так, как задано по-умолчанию. Обычно по-умолчанию задается режим выполнения функций как рекурсивных. Однако это умолчание можно изменять с помощью следующих предложений препроцессора:

```
#pragma nonrec
#pragma recursive
```

Первым предложением по умолчанию задается режим выполнения функций, как не рекурсивных. Второе предложение предписывает рассматривать все функции, как рекурсивные (если явно не оговорено противоположное).

Для случая, когда действует предложение #pragma, функция max() будет выполняться согласно последнему описателю режимов "рекурсивная/не рекурсивная".

Рекомендуется давать явное указание "nonrec" компилятору, если нужно, чтобы он воспринимал функции как не рекурсивные. По такому указанию, компилятор гарантированно сгенерирует объектный код, более короткий и более быстрый, чем в других случаях. В обычных прикладных программах бóльшая часть функций не рекурсивна, поэтому в начале исходного текста следует писать предложения препроцессора: #pragma nonrec

Функции с переменными параметрами

Компилятор SOLID C обрабатывает функции двух типов: с переменным числом параметров и с фиксированным. В функциях первого типа число и представление параметров могут изменяться, тогда как в функциях второго типа они фиксированы.

Например, функции printf, scanf и ряд других являются функциями с переменным числом параметров. В компиляторе SOLID C функции указанных двух типов различаются с помощью специальных описателей.

Структура предложений языка изменяется с введением простых и абстрактных описателей следующим образом:

- С простым описателем:
  идентификатор
  (описатель)
  *описатель
  описатель()
  описатель(.)
  описатель [константное выражение]
- С абстрактным описателем:
  пусто
  абстрактный описатель
  *абстрактный описатель
  абстрактный описатель()
  абстрактный описатель(.)
  абстрактный описатель [константное выражение]

Конструкции "описатель(.)" и "абстрактный описатель(.)" впервые введены для обозначения функций с переменными параметрами. Конструкции "описатель()" и "абстрактный описатель()" обозначают функции с фиксированными параметрами.

Данный метод описания функций с переменными параметрами используется в предложениях типа cast, sizeof. Например, в следующих описаниях функции func и fp определены, как функции с переменными параметрами:

```
int func(.), (*fp)(.);
```

Перед применением функции с переменными параметрами, ее необходимо всякий раз описывать подобным образом. В библиотеке стандартных функций имеются функции с переменными параметрами, как например:

```
printf, fprintf, sprintf, scanf, fscanf, sscanf
```

Описания перечисленных функций содержатся в стандартном файле "stdio.h".

Вызов функции с переменными параметрами совершенно идентичен вызову функции с фиксированными параметрами.

Эти функции не обладают мобильностью с точки зрения рабочей машины (главным образом по методу доступа к параметрам). Отсутствие переносимости имеет место только для самой функции с переменными параметрами. Внешняя функция, ее вызывающая, является переносимой.

Определение функции с переменными параметрами должно быть записано в самом начале, перед ее телом. Ниже приведено типичное определение функции func, являющейся функцией с переменными параметрами:

```
int func(.);        /* это определение обязательно должно идти в начале */

int func(nargs, args)
int nargs, args;
{
  ...
}
```

В момент вызова функции с переменными параметрами, вместо фактического параметра подставляется его копия – осуществляется вызов по значению.

Значением параметра nargs является количество фактических параметров – фиксированная величина. Благодаря ей можно установить, сколько фактических параметров реально передается в функцию с переменными параметрами со стороны вызывающей функции. Однако возможности ограничены тем, что можно знать только число передаваемых параметров. При этом, установить тип каждого из этих параметров невозможно.

Для того, чтобы организовать доступ к указанным параметрам, нужно описать каждый из передаваемых при вызове функции параметров. Первый из параметров (если он имеется) расположен по адресу массива args. Другими словами, значение первого из параметров args присваивается идентификатору args. Второй из параметров помещается по адресу, сдвинутому в сторону роста адресов памяти.

Вообще можно составить последовательность адресов:

адрес 2-го параметра = адрес args + 2
адрес 3-го параметра = адрес args + 4

В следующей таблице приведено соответствие параметров и их адресов:

Номер параметра	Адрес
1	Адрес начала массива args
2	Адрес начала + 2
3	Адрес начала + 4
...	
n	Адрес начала + (n-1)*2

Согласно соответствию адресов и последовательности параметров, приведенному в таблице, можно организовать доступ к этим параметрам. Например, для того, чтобы сослаться на первый и второй параметры типа "char", необходимо написать в программе:

(char)args – ссылка на первый параметр типа "char"
(char)+(&args+1) – ссылка на второй параметр типа "char"

Примечание: *Если в приведенном выше примере args является "int", то благодаря описанию, значение &args преобразуется в тип "int". Адрес следующего по порядку аргумента формируется согласно формуле &args+1, в которой константа "1" преобразуется в соответствии с масштабом разрядной сетки конкретной машины: как правило, это два байта (вообще, можно определить эту величину с помощью выражения sizeof(int*)). В конкретном случае это дает "адрес начала массива параметров плюс два". Аналогичным образом можно организовать доступ к параметрам других типов.*

В заключение данного параграфа приведены два примера простых функций с переменными параметрами: функция sum возвращает значение суммы переменного числа "int" параметров. Функция cmax возвращает максимальное значение из ряда "int" параметров в TINY (char), число которых – переменное.

```
#pragma nonrec
typedef char TINY;

int sum(.);
TINY cmax(.);

int sum(nargs, args)
int nargs, args
{
    int i, *p;
    for(i=0, p=&args; nargs --;)
          i += *p++;
    return i;
}
```

```
TINY cmax(nargs, args)
int nargs;
TINY args;
{
    int *p, max;
    for(max=0, p=(int*)&args; nargs--; p++)
        if(max < (TINY)*p)
            max = (TINY)*p;
    return max;
}
```

Преобразования байтовой арифметики

Для повышения эффективности объектного кода, компилятор SOLID C выполняет преобразования байтовой арифметики. Благодаря этой особенности, все преобразования переменных, имеющих тип "char", выполняются не со словами, а с байтами.

Поскольку процессор Z80 с восьмибитовым форматом команд выполняет эти операции значительно быстрее, нежели другие. Байтовая арифметика особенно эффективна при написании специальных программ, включающих байтовые преобразования.

Результаты логических операций и операций отношения:

```
&&  ||  !  =  !=  >  <  >=  <=
```

имеют тип "char".

Традиционно, те байтовые переменные, которые обозначают внешние кодовые символы, специфицируются с помощью определения типа typedef в виде TINY или BOOL. Первый определяет арифметические объекты, а второй – логические.

Определения указанных типов включены в файл стандартных заголовков "types.h".

Ниже даны несколько примеров употребления байтовых операций, на которые следует обратить особое внимание!

1). Переполнение и исчезновение

При выполнении байтовых арифметических операций, возможны случаи переполнения или исчезновения. Например, в процессе выполнения операторов

```
int i;
TINY x,y;

x = 10; y = 30;
i = x * y;
```

вместо правильного значения 300, переменная "i" получает неправильное значение 44. Это происходит вследствие того, что в процессе выполнения умножения по правилам байтовой арифметики теряется старший байт результата. Для того, чтобы получить правильный результат (число 300), нужно оперировать с целыми операндами, для чего следует специфицировать их с помощью функции cast, как показано в следующем примере:

```
i = (int)x * (int)y;
```

Такое же явление возникает и в следующей ситуации:

```
i = x << 8 | y;
```

При выполнении операции сдвига над операндом байтовой длины, может быть потеряна его часть вследствие переполнения. На это также нужно обратить внимание. Величину

"y" необходимо подставить в выражение для "i" в том же самом виде, что и показано ниже:

```
i = (unsigned)x << 8 | y;
```

2). У типа TINY (char) отсутствует знак числа. Переменные типа "char" используются без знака. Поэтому иногда возникают ситуации, в которых результат операции вычитания становится отрицательным. Поэтому, если выполнить нижеследующую программу, в переменную "i" будет записано 255:

```
int i;
TINY x,y;

x = 2;
y = 3;
i = x - y;
```

Для того, чтобы получить правильный результат, нужно выполнить эту операцию в соответствии с типом "int":

```
i = (int)x - (int)y;
```

3). Согласование параметров

Компилятор SOLID C проводит четкое различие между типами "int" и "char".

Поэтому фактические и формальные параметры должны быть обязательно согласованы между собой по типу. Следующий пример иллюстрирует неправильный вызов функции:

```
func(x)
TINY x;
{
   ...
}
main()
{
   func(1);    /* неправильный вызов функции */
}
```

Нужно изменить вызов следующим образом:

```
func((TINY)1);
```

Согласование параметров по типу, компилятором не проверяется. Поэтому, следует обращать особое внимание на подобные ситуации.

Распределение регистров

Поскольку компилятор SOLID C поддерживает распределение регистровых переменных, он способен генерировать более эффективный объектный код. В стандартном языке Си предусмотрено размещение непосредственно в регистрах (класс памяти "register") только нескольких первых переменных из всего их множества.

(Какое именно количество регистровых переменных будет помещено в регистры, зависит от того, какая рабочая машина будет использоваться). В отличие от стандартного, компилятор SOLID C автоматически выполняет оптимальное распределение регистров, в соответствии с потоком данных. Этим самым снята необходимость обязательного указания переменных программы, как регистровых.

Вопрос, какие из переменных будут помещены в регистры, решается на уровне кодогенерации. Поэтому описание "register" идентично по смыслу описанию "auto" и никакого другого смысла не имеет.

В каждой функции программы, первые 16 переменных простого типа становятся кандидатами на размещение в регистрах, как переменные типов "auto" или "register". К числу таких простых типов относятся "int", "char", "unsigned" и "указатель".

Средствами языка Си можно получать адреса переменных. Компилятор SOLID C также, безусловно, располагает этими средствами. Однако, в случаях тех переменных, которые являются объектами распределения регистровой памяти, возникает проблема. Она обусловлена тем, что как таковой "адрес" регистра не существует.

Например, если написать

```
int n;
n = 10;
scanf("%d", &n);
printf("%d", n);
```

и ввести 100, то какое значение получит переменная "n" ?. Конечно, 100.

Компилятор поместит значение переменной по заданному адресу в память, если указать ему значение этого адреса. Следовательно, никаких проблем не возникнет, если при вызове функции передать ей в качестве параметра адрес переменной.

Однако в примере

```
 int n;
 int *p;
 p = &n;
 n = 10;
*p = 100;
 printf("%d", n);
```

возможна ситуация, когда в качестве значения переменной "n" будет индицировано не 100, а 10. Между двумя предложениями, одно из которых "n=10", а другое "printf("%d",n)", не фигурирует "&n". Поэтому компилятор не помещает "n" в память.

НЕКОТОРЫЕ ДРУГИЕ ОСОБЕННОСТИ КОМПИЛЯТОРА

Таблица представления данных различного типа

Тип	Разрядность	Диапазон значений
char	8	0..255
short	16	-32768..32767
int	16	-32768..32767
unsigned	16	0..65535

1. В рамках компилятора SOLID C проводится идентификация переменных и функций по первым 30-ти символам имени. Благодаря такому расширению имен, можно использовать легко поддающиеся анализу имена переменных и функций, и получать удобочитаемые тексты программ.

2. Позиция ошибки диагностируется по номеру ошибочной строки и начальному символу (колонки в строке). Начальная позиция считается с 0. Таким образом, позиция первого символа программного текста есть "нулевая строка, нулевая колонка".

3. Для указания имени включаемого файла используется предложение "#include" препроцессора, как описано ниже:

<имя файла> – ссылка на файл, расположенный в подкаталоге "INCLUDE" каталога компилятора.

"имя файла" – ссылка на файл, расположенный по текущему пути исходного ".c" файла.

Задание пути в имени файла не допускается. Например, если имеются предложения

```
#include <stdio.h>
#include "common.h"
```

то в первом случае сделана ссылка на стандартный файл компилятора Си, расположенного в подкаталоге с зарезервированным именем "INCLUDE" каталога SOLID C (по-умолчанию "SOLID"), а во втором случае - на файл, расположенный по текущему пути входного ".c" файла.

Следует обратить внимание на то, что имя подкаталога "INCLUDE" зарезервировано и его не следует менять.

4. Типы long, float, double не поддерживаются, но зарезервированы. Поэтому нельзя использовать слова "long", "float" и "double" в качестве идентификаторов переменных и меток.

5. Не поддерживаются битовые поля структур.

6. Некоторые ограничения в константных выражениях. Нельзя включать в константные выражения указанных ниже типов унарную операцию "sizeof":

a) сразу после оператора "case" (в качестве метки);
b) в описателе массива в качестве его размерности.
Подобные ограничения не действуют на те константные выражения, которые появляются в инициаторе (т.е. при инициализации).

7. Имеется отличие от стандартного языка Си по эффективным пределам имен членов структур и объединений (struct & union). В стандартном Си одно и то же имя члена может появляться в двух различных структурах только в случае, когда они оба одного типа и имеют одинаковое смещение в обеих структурах. Компилятор SOLID C воспринимает имя члена так, как принято в Паскале или в Аде. Таким образом, одно и то же имя члена может появляться в разных структурах и будет правильно обработано компилятором SOLID C.

В качестве примера рассмотрим следующее описание:

```
struct node {
       char *word;
       int count;
       struct node *next;
} pool[1000], *p;

struct noad {
       int atr;
       struct noad *next;
} table[10], *q;
```

В стандартном языке Си данная запись является неправильной, потому, что имя члена "next" появляется в двух структурах - "node" и "noad", где их типы и смещения не согласованы. Однако в среде компилятора SOLID C эти записи являются правильными.

Следует обратить внимание на следующую запись

```
p->next
```

которая описывает член типа "struct node" (потому что "p" есть указатель типа "struct node"). Благодаря такому приему можно подбирать правильные члены путем строгого указания их типов. Так, в операциях "." и "->" левые члены должны быть указателями структур, включающих члены, записанные в качестве правой части этих операций. Например:

```
i->count
```

включает "i" не как указатель типа "struct node", а как "int" (целое). Поэтому компилятор SOLID C такую ситуацию не диагностирует. Для того, чтобы получить такой же результат, необходимо переписать приведенное выражение следующим образом:

```
((struct node *)i)->count
```

В приведенном выше примере для того, чтобы операция "->" была более высокого ранга, нежели операция "cast", следует терм

```
(struct node *) i
```

заключить в скобки.

До того, пока не будет преобразован тип указателя в "int", что выполняется с помощью функции cast, операция не может быть выполнена.

Даже при выполнении арифметических операций в случае указателей разных типов, также необходимо использовать функцию cast. Например, в случае описаний

```
int i;
char *p;
int *q;
```

операции

```
p = i;
if(p == i)
  q = p;
if(p == q)
```

будут отмечены как ошибочные.

Две первые ошибочны потому, что производятся подстановки между типами указателя и "int" (целого), и операции сравнения между ними. Две последние также ошибочны, потому, что несмотря на то, что и "p", и "q" обе являются указателями, но они специфицированы разными типами и поэтому компилятор отметит их, как ошибочные.

Эти ошибки можно устранить, либо использованием функции cast, либо указанием опции "-k" на первом проходе компилятора.

С другой стороны, разрешены преобразования типов из целого в указатель и обратно. Это обусловлено тем, что в языке Си целая константа 0 используется как указатель особого адреса. Например, запись

```
char *p;
if(p != 0)
  p = 0;
```

не содержит ошибки.

Типы "int" и указателей нельзя смешивать в среде компилятора SOLID C. Данное соглашение вообще может считаться показателем хорошего стиля программирования, при котором отладка программы становится проще.

8. Нет автоматического преобразования между типами "указатель" и "int", и обратно. В стандартном языке Си разрешены операции сравнения значений указателей и целых (int), но в SOLID C смешивание указанных типов вызовет сообщение об ошибке. Однако этот режим можно разрешить, указав опцию компилятора "-k" (на первом проходе).

9. Тип "char" интерпретируется как значение без знака, т.е. может иметь значение 0..255.

10. В рамках стандартного языка Си, в случаях появления не объявленных ранее функций, они интерпретируются как возвращающие "int" значение. Параметры не объявленного типа, которые вошли в список параметров, считаются "int".

Рассматриваемый здесь компилятор не позволяет использовать эти умолчания. То есть, хороший стиль программирования предполагает описание всех идентификаторов до того, как на них будут выполняться ссылки - автоматические ссылки выполняться не будут.

Все объявления, которые относятся к функциям стандартной библиотеки, записаны в файле стандартных заголовков, поэтому любую из стандартных функций можно использовать без объявления. Поэтому все пользовательские программы должны содержать ссылку на файл "stdio.h" в самом начале текста, в предложении #include <stdio.h>.

Однако, если на первом проходе компилятора указать опцию "-k", указанные описания функций будут объявлены по-умолчанию. От этого теряется качество получаемого кода, поэтому данной опцией целесообразно пользоваться, если важно обеспечить совместимость со стандартным языком Си.

11. В данной версии рассматриваемого компилятора не поддерживается предложение препроцессора "#if". Вполне возможно заменить указанное предложение другими: "#ifdef" или же "#ifndef". Возможны также другие варианты замены предложения "#if". Рассматриваемый здесь компилятор не генерирует код тех участков программы, выполнение которых не требуется при ее прогоне. Поэтому можно получить точно такой же результат, если применить оператор "if" вместо предложения "#if". Например, можно заменить текст одной из нижеследующих программ на другую:

Программа 1:

```
#if sizeof(FCB) != sizeof(char[36])
    Open_MSX();
#else
    Open_OTHER();
#endif
```

Программа 2:

```
if (sizeof(FCB) != sizeof(char[36]))
   Open_MSX();
else
   Open_OTHER();
```

12. Введена директива препроцессора "#pragma". Применяется для различных указаний компилятору. Все предложения данного вида могут появляться в любом месте исходной программы. Ниже, в качестве примеров приведены четыре предложения с заданием соответствующих директив компилятору:

1. #pragma optimize time

Данное указание компилятор интерпретирует так, что он старается сгенерировать объектный код, отличающийся более высокой скоростью выполнения, нежели в случае, когда он должен быть более компактным.

2. #pragma optimize size

При таком указании, компилятор считает более важным сгенерировать компактный объектный код, хотя бы и не очень быстрый в исполнении.

3. #pragma nonrec

Этой директивой изменяется задание по-умолчанию режима выполнения функций, как не рекурсивных.

4. #pragma recursive

С помощью данного предложения устанавливается по-умолчанию режим выполнения функций как рекурсивных.

13. Поскольку компилятор SOLID C поддерживает для переменных автоматическое распределение регистров, класс памяти всегда устанавливается "auto" (автоматический).

14. Компилятор SOLID C позволяет гнездовать тексты комментариев. Эта функция обеспечивает возможность написания больших программ, содержащих комментарии, в теле которых, в свою очередь содержатся комментарии. Если на первом проходе указать компилятору опцию "-c", гнездование комментариев производиться не будет, что отвечает возможностям стандартного языка Си.

ПЕРЕДАЧА ПАРАМЕТРОВ

Функции с фиксированными параметрами

В случае обычных функций (функций с фиксированными параметрами), первые три параметра помещаются в регистры, а все последующие – в стек. Если какой-либо из 3-х параметров имеет тип "char", он помещается в соответствующий 8-ми битный регистр. Параметры (первые три) других типов помещаются в регистровые пары.

4-й параметр (если есть) и все последующие – помещаются в стек, в обратном порядке (паскалевский способ ?). На вершину стека помещается адрес возврата.

Каждый из размещаемых в стеке параметров занимает два байта. Значение "char" параметра помещается в младший байт, а значение старшего байта остается неопределенным.

В режиме совместимости со стандартным языком Си, использование параметра типа "char" имеет отличия: преобразование из "char" формата в "int" производится автоматически (при этом старший байт обнуляется), после чего данные рассматриваются как "другой тип".

Тип параметра	Первый	Второй	Третий	Четвертый	Пятый	...
char	A	E	C	(SP+2)	(SP+4)	...
другой тип	HL	DE	BC	(SP+2) (SP+3)	(SP+4) (SP+5)	... ...

Функции с переменными параметрами

Когда вызывается функция с переменными параметрами, все параметры заталкиваются в стек (в обратном порядке). Число параметров, помещенных в стек, записывается в регистровую пару HL. На вершину стека помещается адрес возврата.

В "собственном" режиме, "char" параметр помещается в младший байт, значение старшего байта остается неопределенным.

В режиме совместимости со стандартом языка Си, "char" тип автоматически преобразуется в "int" тип (при этом старший байт обнуляется), который далее трактуется как "другой тип".

Параметры других типов размещаются в стеке согласно правилам стандарта для процессора Z80, а именно: первым запоминается младший байт, вторым - старший.

Тип параметра	Количество	Первый	Второй	Третий	...
char	HL	(SP+2)	(SP+4)	(SP+6)	...
другой тип	HL	(SP+2) (SP+3)	(SP+4) (SP+5)	(SP+6) (SP+7)	... ...

ВОЗВРАТ ЗНАЧЕНИЯ ФУНКЦИИ

При передаче управления вызывающей программе, функция может передавать свое значение через регистры. При этом функция типа "char" передает значение через регистр "A". Функции других типов передают свои значения через регистровую пару HL.

В режиме совместимости со стандартным языком Си, возвращаемое значение всегда передается через регистровую пару HL, даже в том случае, когда функция имеет "char" тип.

Для возврата в вызывающую функцию двух и более значений, необходимо использовать указатели переменных, как параметры любых других си-программ.

ПРАВИЛА ВЫЗОВА ФУНКЦИЙ

1. Во-первых, вызывающая программа оформляет параметры и помещает их в регистры или заталкивает в стек согласно правилам, описанным выше.

2. Программа вызывает функцию. Для вызова обычно используется слово "call".

3. После возврата управления функцией, параметры выталкиваются из стека (с использованием команды "pop"). Если все параметры размещены в регистрах, использовать команду "pop" нет необходимости. Следует отметить, что в случае применения команды "pop", возвращаемые через регистры "A" или "HL" значения функций не будут потеряны.

4. Значение функции возвращается через регистр "A", если функция имеет "char" тип. Для функций другого типа, значение всегда возвращается через регистровую пару HL.

ПЕРЕДАЧИ УПРАВЛЕНИЯ ВЫЗЫВАЕМОЙ ФУНКЦИИ

Вызываемая функция не должна разрушить содержимое указателя стека. Иными словами, содержимое указателя стека при выходе из функции должно остаться тем же,

что и при входе в нее. Параметры, загруженные в стек, должны выталкиваться из него вызывающей программой. Вызываемая функция может изменить значения параметров (как тех, которые передаются через регистры, так и загруженных в стек).

Однако следует подчеркнуть, что в языке Си действует соглашение, в рамках которого вызов функций осуществляется по значению, поэтому даже если будут возвращены измененные параметры, это никакого влияния на процесс вызова функции не окажет.

Возвращаемое значение функции помещается в регистр "A" или в регистровую пару HL – в зависимости от типа вызванной функции. Эти правила описаны в предыдущем разделе.

Пример 1:

В следующем примере объявлена ассемблерная функция ISDIGIT, которая возвращает значение функции BOOL в регистре "A". Вызывающая функция передает параметр "char" типа через регистр "A".

... вызывающая программа (на языке Си)...

```
typedef char BOOL;       /* определение "char" типа как BOOL */
...
BOOL isdigit(c);
char c;
  ...
  if (isdigit(c))
  ...
  ...
```

... вызываемая функция (на языке Ассемблера)...

```
         public isdigit_

isdigit_: cp   '0'
          jr   c,false
          cp   '9'+1
          ret  c           ; цифра, вернуть true
false:    xor  a           ; не цифра, вернуть false
          ret
```

Пример 2:

Этот пример соответствует случаю вызова функции, написанной на Си, из ассемблерной процедуры.

Стандартные библиотечные функции printf и puts вызываются из ассемблерной процедуры. Их исходные тексты находятся в исходниках библиотеки, поэтому здесь функции printf и puts не будут описаны. Следует заметить, что поскольку функция printf – с переменными параметрами, ее параметры заталкиваются в стек.

В данном примере параметр всего один, однако в случаях двух и большего числа параметров, они помещаются в стек в обратной последовательности. Число параметров передается в вызываемую функцию через регистровую пару HL (в приведенном примере передается число 1). После возврата из вызываемой функции, будут выполняться операции выталкивания из стека (в примере команда "pop" будет выполнена 1 раз). Следует помнить, что параметры восстанавливаются из стека вызывающей программой, а не вызываемой.

С другой стороны, функция puts принадлежит к числу функций с фиксированными параметрами, поэтому адрес символьной строки передается через регистровую пару HL.

```
         extern  printf_, puts_
```

```
example: ld     hl,msg
         push   hl          ; поместить в стек адрес символьной строки
         ld     hl,1        ; загрузить число параметров
         call   printf_     ; вызов процедуры, формирующей вывод
         pop    hl          ; выталкивание параметров из стека
         ;
         ld     hl,msg      ; загрузить адрес строки
         jp     puts_       ; вызов функции вывода строки

msg:     db     'Hello, world!',0Ah,0
```

РЕКОМЕНДАЦИИ ПО ПРОГРАММИРОВАНИЮ

1. Следует пользоваться не рекурсивными функциями, если нет особой необходимости использовать рекурсию. Благодаря такому подходу, можно достичь более быстрого доступа к тем локальным переменным, для которых не распределяется регистровая память и следовательно, получить эффективный объектный код.

2. Если указан режим совместимости со стандартным языком Си (опция "-k"), то качество объектного кода будет ниже, поэтому указание на этот режим нужно применять только в случаях, когда требуемая совместимость на уровне стандартного языка Си нужна больше, чем эффективность кода.

3. Используйте тип "char" (с 8-битовой длиной данных) везде, где только можно. Байтовые операции выполняются с помощью 8-битового процессора более эффективно, чем над данными, длиной в слово (типы "int" или "unsigned").

Поэтому определяйте переменные, числовые значения которых не выходят за пределы диапазона 0..255, как TINY (char).

4. При использовании 16-битовых переменных следует определять их преимущественно типом "unsigned", который в этих случаях предпочтительнее, нежели "int".

Это обусловлено тем, что в процессоре Z80 выполнение операций со знаком (в частности, операций сравнения величин со знаком) реализовано не лучшим образом.

5. Нежелательно установление статического режима для локальных переменных.

Переменные, определенные как статические или внешние, не подлежат загрузке в регистровую память. Поэтому, если для повышения скорости выполнения программы определить локальные переменные как "static", а функции как "extern", то результат будет противоположным - эффективность объектного кода существенно понизится. Компилятор SOLID C сам производит распределение регистров, поэтому следует обязательно определять локальные переменные, как "auto".

ЗАРЕЗЕРВИРОВАННЫЕ СЛОВА ЯЗЫКА

Следующие слова используются как ключевые в компиляторе SOLID C и их нельзя использовать в качестве меток или имен переменных.

asm	float	signed
auto	for	static
break	goto	struct
case	if	switch
char	int	typedef
const	long	union
continue	nonrec	unsigned
default	recursive	void
do	register	volatile
double	return	while
else	short	
extern	sizeof	

ДИРЕКТИВЫ КОМПИЛЯТОРА

```
define          nonrec
endif           optimize
error           pragma
ifdef           recursive
ifndef          size
include         time
line            undef
```

ОПИСАНИЕ ФУНКЦИЙ БИБЛИОТЕКИ СИ

Последняя редакция: 3.09.2004

Примечание.

Для функций форматного ввода/вывода, строка указанного формата не преобразуется и в выходной поток не передается, а становится спецификатором преобразований, выполняемых по следующим правилам:

% [-] [[o]w] [.n]c

Знак "-" указывает на то, что данное поле подвергается сдвигу влево (обычно поля сдвигаются вправо).

Знак "w" обозначает максимальное ограничение поля по ширине. Если перед знаком "w" поставлен нуль, то данное поле в отличие от пробела имеет символ расширения "нуль".

Знак "-n" действует только в том случае, когда имеется преобразователь строк (%s). С помощью знака ".n" устанавливается максимальное число символов, выводимых в одной строке.

Символ "c" может принимать одно из следующих значений:

d - десятичное число со знаком;

u - десятичное число без знака;

o - восьмеричное число без знака;

x - шестнадцатиричное (без знака);

s - символьная строка;

c - символ.

Информация, в составе которой отсутствуют приведенные выше символы преобразований, передается в выводимый поток без изменений. Например, с помощью последовательности "%%" можно вывести один символ процента "%".

ABORT

```
#include <stdlib.h>

void abort()
```

Описание.

Функция выводит на экран сообщение "Abnormal program termination" (непредвиденное завершение программы) и завершает вызванный процесс без обновления буферов файлов.

ABS

```
#include <stdlib.h>

int abs(int n);
```

Описание.

Функция abs возвращает значение n, равное абсолютной величине n.

ABSREAD

```
#include <dos.h>
```

```
char absread(char disk, int nsect, char count, char buffer)
```

disk – номер диска

nsect – номер начального сектора

count – кол-во читаемых секторов

buffer – адрес буфера под считываемые данные

Описание.

Абсолютное чтение с диска. Функция считывает в буфер один или несколько секторов с заданным начальным относительным номером. Секторы нумеруются с 0 от начала логического (не физического!) диска. К примеру, загрузочный сектор данного логического диска имеет номер 0, далее первая копия FAT начинается с сектора номер 1 и т.д.

Функция возвращают 0 при успешном завершении операции. В случае ошибки возвращается -1 (0FFh) и errno содержит номер ошибки.

ABSWRITE

```
#include <dos.h>
char abswrite(char disk, int nsect, char count, char buffer)
```

disk – номер диска

nsect – номер начального сектора

count – кол-во записываемых секторов

buffer – адрес буфера данных

Описание.

Абсолютная запись на диск. Функция записывает из буфера один или несколько секторов с заданным начальным относительным номером. Секторы нумеруются с 0 от начала логического (не физического!) диска. К примеру, загрузочный сектор данного логического диска имеет номер 0, далее первая копия FAT начинается с сектора номер 1 и т.д.

Функция возвращают 0 при успешном завершении операции. В случае ошибки озвращается -1 (0FFh) и errno содержит номер ошибки.

ATEXIT

```
#include <stdlib.h>
char atexit(*func)
```

Описание.

Функция регистрирует (заносит в свой список) функцию *func как "exit function", т.е. отрабатывающую перед выходом в ДОС. В случае нормального окончания программы, exit вызывает (без аргументов) "exit function" как раз перед возвратом в операционную систему. Каждый вызов atexit регист рирует другую завершающуюся функцию. Регистрируемые функции не могут иметь параметров. Может быть зарегистрировано не более 8 функций, которые выпол няются в порядке "последний пришел - первый ушел" (LIFO).

Функция atexit соответствует стандарту ANSI и должна использоваться вместо onexit, если требуется совместимость с ANSI.

Функция возвращает 0, если все нормально и ненулевое значение при ошибке (если уже определено макс. число exit-функций).

Пример

```
main()
{
   atexit(func1);        // "Exit Function 1 called"
   atexit(func2);        // "Exit Function 2 called"
   printf("Main quitting ...\n");
}
```

Выведется:

```
Main quitting ...
Exit Function 2 called
Exit Function 1 called
```

ATOI

```
#include <stdlib.h>
int atoi(*string)
```

Описание.

Функция преобразует символьную строку string в целое значение. Исходная строка string представляет собой последовательность символов, которые могут быть интерпретированы как числовое значение определенного типа. Функция прекращает чтение исходной строки, как только появится первый символ, который не может быть воспринят как часть числа (им может быть нулевой символ, завершающий строку).

Функция возвращает значение int типа.

Возвращаемое значение равно 0, если вход не может быть преобразован в значение данного типа. В случае переполнения, возвращаемое значение не определено.

BDOS

```
#include <dos.h>
char bdos(char n, int doshl, char dosa)
int bdosh(char n, int doshl, char dosa)
```

Описание.

Функция выполняет системный вызов ДОС, номер которого определен в n. Аргументы doshl и dosa размещаются соответственно в регистрах HL и A.

Функция выполняет команду RST 10h для осуществления системного вызова.

Функция может быть использована только для системных ДОС-вызовов, которые не принимают аргументов или принимают их только в регистрах HL и A.

Функция возвращает значение регистра A, заполненного системным вызовом.

Примечание: Эту функцию нельзя использовать для осуществления системных вызовов, в которых ошибки указываются посредством установки флага переноса.

Т.к. си-программы не имеют доступа к этому флагу, статус возвращаемого значения определить невозможно. В этом случае необходимо использовать функцию intdos.

Функция bdosh отличается от bdos только тем, что возвращаемое значение содержится в рег. паре HL.

Обе функции можно использовать для ДОС-вызовов: 02h, 0Ah, 0Bh, 0Eh, 11h, 1Bh, 1Ch, 1Dh, 1Eh, 5Ch функций.

BSEARCH

```
#include <stdlib.h>

void *bsearch(*key, *base, num, size, *compare)

      char *key;          -  указатель на ключ (строку) поиска

      char *base;         -  указатель на массив

      unsigned num;       -  число элементов массива

      unsigned size;      -  размер каждого элемента

      int (*compare)();   -  указатель на функцию сравнения
```

Описание.

Функция bsearch выполняет двоичный поиск в отсортированном массиве. Аргумент compare является указателем на процедуру, поставляемую пользователем. В процессе поиска, функция bsearch может вызывать эту процедуру один или несколько раз, при каждом вызове передавая указатели на два элемента массива. Процедура compare должна сравнивать элементы, а затем возвращать одно из следующих int-значений:

```
< 0      element1 < element2

  0      element1 = element2

> 0      element1 > element2
```

Функция bsearch возвращает указатель на первое вхождение ключа key в массиве.

Возвращается NULL, если ключа key в массиве нет.

CALLOC

```
#include <malloc.h> или <stdlib.h>

char *calloc(int n, int size)
```

Описание.

Функция захватывает пространство для хранения массива из n элементов, каждый размером size байт. Каждый элемент инициализируется в 0.

Функция возвращает указатель на захваченное пространство.

Если недостаточно памяти, возвращается значение NULL.

CGETS

```
#include <conio.h>

char *cgets(*str)
```

Описание.

Функция читает строку символов прямо с консоли, запоминает строку и ее длину в аргументе str. Аргумент str должен быть указателем на массив символов. Первый элемент массива - str[0] должен содержать максимальную длину (в символах) читаемой строки. Массив должен быть достаточно большим, чтобы вместить строку и заключительный нуль. Функция продолжает считывать символы, пока не будет прочитан код новой строки (0x0D) или заданное количество символов. При этом, код новой строки

заменяется на нуль. Строка запоминается с str[2] элемента массива. Во втором элементе массива str[1] запоминается действительная длина строки.

Функция возвращает указатель на начало строки, находящейся в str[2].

В случае ошибки, возвращаемое значение не определено.

CHDIR

```
#include <direct.h>

char chdir(char *pathname)
```

Описание.

Функция изменяет текущий каталог и текущий диск, если он указан в pathname. Если путь начинается с "\" - это означает путь от корневого каталога, иначе от текущего. Pathname должен ссылаться на существующий каталог.

Функция возвращает значение 0, если рабочий каталог успешно изменен.

При ошибке, возвращается -1 (0FFh) и errno содержит номер ошибки.

CLEARERR

```
#include <stdio.h>

void clearerr(FILE *fp)
```

Описание.

Функция clearerr относится к высоко-уровневому вводу/выводу. Функция сбрасывает в FILE-структуре файла 4-й бит "индикатора ошибки" и 5-й бит "индикатора конца файла". Индикаторы ошибок автоматически не очищаются: если для файла установлен индикатор ошибки, то операции над файлом будут возвращать значения ошибок до тех пор, пока не будут вызваны функции clearerr или rewind.

CLRSCR

```
#include <conio.h>

void clrscr()
```

Описание.

Функция производит очищение экрана.

CPRINTF

```
#include <conio.h>

int cprintf(char *format, arg1, arg2, ...)
```

Описание.

Функция выводит с преобразованием формата наборы символов и значений прямо на экран. Каждый аргумент должен указывать на значение с типом, который соответствует типу, заданному в строке формата.

Функция возвращает число выведенных символов. При ошибке вывода, функция возвращает отрицательное значение.

CPUTS

```
#include <conio.h>

void cputs(char *str)
```

Описание.

Функция выводит текстовую строку прямо на экран. Строка должна оканчиваться нулем.

CREAT

```
#include <io.h>

uint creat(char *name, int attr)

uint creat(char *name, char attr)
```

Описание.

Функция низко-уровневого ввода/вывода. Функция creat создает новый файл. Если файл уже существует, урезает его до нулевой длины. Значение attr состоит из одной или нескольких констант:

S_IWRITE и S_IREAD, объявленных в "stat.h". Если задаются обе константы, они объединяются при помощи логического ИЛИ "|".

Функция возвращает номер дексриптора.

При ошибке, возвращает значение -1 (0FFFFh) и errno содержит номер ошибки.

Функция возвращает номер дескриптора файла, если Ok.

При ошибке, возвращает значение -1 (0FFFFh) и ячейка errno содержит номер ошибки.

DEC8

```
#include <stdlib.h>

void dec8(char value)
```

Описание.

Функция выводит аргумент value в десятичном формате, без ведущих нулей. Диапазон выводимых значений 0..255. Функция не возвращает никакого значения.

DEC16

```
#include <stdlib.h>

void dec16(uint value)
```

Описание.

Функция выводит аргумент value в десятичном формате, без ведущих нулей. Диапазон выводимых значений 0..65535. Функция не возвращает никакого значения.

DEC32

```
#include <stdlib.h>

void dec32(uint high, uint low)
```

Описание.

Функция выводит два 16-ти битных аргумента high (ст.разряд) и low (мл.разряд) как одно значение. Вывод происходит в десятичном формате, без ведущих нулей. Диапазон выводимых значений 0...4 млрд.

Функция не возвращает никакого значения.

DISABLE

```
#include <dos.h>

void disable()
```

Описание.

Функция запрещает прерывания.

DIV

```
#include <stdlib.h>

struct divt *div(uint num1, uint num2)
```

Описание.

Функция делит num1 на num2, возвращая частное и остаток. Возвращает структуру типа div_t, определенную в <stdlib.h>.

ENABLE

```
#include <dos.h>

void enable()
```

Описание.

Функция разрешает прерывания.

_EXIT

```
#include <stdlib.h>

void _exit(int code)
```

Описание.

Функция завершает вызванный процесс без обновления буферов потока. Значение code обычно устанавливается в 0 для указания нормального выхода и устанавливается в любое другое значение для определения ошибки. Поскольку вызов _exit не возвращает значения, то мл. байт code является кодом возв рата для родительского процесса.

EXIT

```
#include <stdlib.h>

void exit(int code)
```

Описание.

Функция завершает вызванный процесс. Перед завершением процесса, функция сбрасывает все буферы в файлы и закрывает все открытые файлы. Перед выходом в операционую систему, функция предварительно вызывает на отработку зарегистрированные функции "exit function" (отрабатывающие перед выходом в ДОС).

Значение code обычно устанавливается в 0 для указания нормального выхода и устанавливается в любое другое значение для определения ошибки.

Поскольку вызов exit не возвращает значения, то мл. байт code является кодом возврата для родительского процесса.

FCLOSE

```
#include <stdio.h>

char fclose(FILE *fp)
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Функция сбрасывает буфер в файл и закрывает его. Данной функцией можно закрывать только те файлы, работа с которыми производится в рамках высоко-уровневого ввода/вывода. Для закрытия файлов в рамках низко-уровневого ввода/вывода, необходимо применять функцию close.

Функция возвращает 0 при успешном завершении. При ошибках возвращается значение -1 (0FFh) и errno содержит номер ошибки.

FCLOSALL

```
#include <stdio.h>

void fclosall()
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Функция сбрасывает все буферы в файлы и закрывает все открытые файлы.

FDOPEN

```
#include <stdio.h>

FILE *fdopen(FD fd, char *mode)
```

Описание.

Функция разрешает файлу, открытому для низко-уровневого ввода/вывода, быть буферированным и форматированным, т.е. преобразует его для высоко-уровневого ввода/вывода. После этого, для всех последующих операций с файлом, необходимо использовать функции высоко-уровневого ввода/вывода. В следствии этого, закрывать файл необходимо функцией fclose, а не close. Символьная строка mode определяет тип доступа к файлу:

r,w,a	-	режим текст. файла
rb,wb,ab	-	режим. бинарного файла
r+, w+, a+	-	r/w, "на обновление" текст. файла
r+b, w+b, a+b	-	r/w, "на обновление" бинарного файла

В добавление к вышеприведенным значениям, может быть добавлен символ "b" или "t", для задания режима трансляции новых строк. Опция "t" не относится к ANSI-стандарту для fopen. Это расширение Microsoft и оно не должно использоваться, если требуется ANSI-совместимость.

Если "b" или "t" не заданы в строке "mode", режим трансляции определяется как "t".

Функция возвращает указатель на открытую FILE-структуру. При ошибке, возвращается NULL.

FFIRST, _FFIRST

```
#include <dos.h>

char _ffirst(char *pathname, FIND *ffblk, char attr)

char ffirst(char *pathname, FIND *ffblk, char attr)
```

Описание.

Функция ffirst производит поиск заданного в pathname файла из группы файлов. Аргумент pathname может содержать имя диска и каталог для поиска. В имени файла допускаются глобальные символы ? и *. Имя найденного файла находится в формате DOS "filename.ext".

Функция _ffirst отличается от функции ffirst тем, что возвращает имя найденного файла в формате "FilenameExt".

Если файл обнаружен, функция заполняет структуру (определенную в "dos.h"), на которую указывает аргумент ffblk.

Аргумент attr содержит атрибут(ы) файлов для поиска и может принимать следующие значения, определенные в "dos.h":

FA_NORMAL	Обычный файл
FA_RDONLY	Только чтение
FA_HIDDEN	Скрытый файл
FA_SYSTEM	Системный файл
FA_LABEL	Метка тома
FA_DIREC	Каталог
FA_ARCH	Архивный

Можно задавать кратные значения атрибутов, используя символ "|" (оператор OR).

Функция возвращает 0 при успешном завершении поиска.

Если файл не найден или в имени файла встретилась какая-либо ошибка, возвращается -1 и errno содержит номер ошибки.

FFLUSH

```
#include <stdio.h>

char fflush(FILE *fp)
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Функция освобождает (записывает) буфер открытого файла. Применяется для сохранения данных на диске, перед выполнением операции, могущей сломать систему.

Если файл открыт для записи, буфер сбрасывается в файл. Если файл открыт для чтения, буфер очищается. После вызова функции, файл остается открытым.

Функция возвращает 0, если буфер успешно обновлен. Это же значение возвращается, когда поток не имеет буфера или когда он открыт только для чтения. В случае возникновения ошибки, возвращается значение EOF.

FGETC

```
#include <stdio.h>

char fgetc(FILE *fp)
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Функция читает текущий символ из потока fp и ставит указатель (если он есть) на следующий символ. Возвращается считанный символ.

В случае возникновения ошибки или достижения конца файла, возвращается значение EOF. Однако, значение EOF может быть воспринято как целая величина, тогда нужно использовать функции ferror и feof для проверки ошибки или конца файла.

FGETPOS

```
#include <stdio.h>

char fgetpos(FILE *fp, fpos_t *position)
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Функция сохраняет текущее положение указателя файла в объекте, на который указывает position. Позднее, функция fsetpos использует значение, хранимое в position для переустановки указателя в файле в новое положение. Значение position хранится во внутреннем формате и оно должно использоваться только с функциями fgetpos и fsetpos.

Функция возвращает 0. При ошибке, возвращается -1.

FGETS

```
#include <stdio.h>

char *fgets(char *string, int maxlen, FILE *fp)
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Функция читает строку из входного потока fp и располагает ее в string.

Символы считываются, начиная с текущей позиции fp и до символа новой строки '\n' или до конца потока, либо пока число считанных символов не станет равным maxlen-1. Строка размещается в string, затем в ее конец ставится нуль. Символ новой строки также включается в string. Если в начале считывания maxlen равнялось 1, то string будет пустой "".

Функция возвращает указатель на строку string. При ошибке или конце файла,

возвращается NULL.

FNEXT

```
#include <dos.h>

char fnext(FIND *ffblk)
```

Описание.

Функция производит поиск следующего совпадающего файла. При каждом вызове, функция возвращает одно имя файла, до тех пор, пока параметры файла будут подходить под заданные параметры поиска. Аргумент ffblk должен указывать на структуру (определенную в <dos.h>), предварительно заполненную функцией ffirst.

Функция возвращает 0 при успешном завершении поиска. Если файл не найден или в имени файла встретилась какая-либо ошибка, возвращается -1 и errno содержит номер ошибки.

FOPEN

```
#include <stdio.h>

FILE *fopen(pathname, mode)
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Функция открывает файл с именем pathname. Символы в строке mode определяют тип доступа к файлу.

Функция возвращает указатель на структуру открытого файла. При ошибке, возвращается NULL.

FPRINTF

```
#include <stdio.h>

int fprintf(FILE *fp, char *format, arg1, arg2, ...)
```

Описание.

Функция выводит данные с преобразованием формата в файл *fp из переменных, определяемых указателями аргументов args. Каждый аргумент должен указывать на значение с типом, который соответствует типу, заданному в строке формата.

Функция возвращает число выведенных символов. При ошибке вывода, функция возвращает отрицательное значение.

FPUTC

```
#include <stdio.h>

char fputc(char ch, FILE *fp)
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Записывает символ ch в файл fp. Функция возвращает записанный символ.

При ошибке, возвращается значение EOF.

FPUTS

```
#include <stdio.h>

char fputs(char *string, FILE *fp)
```

Описание.

Функция копирует строку в конец файла fp. Нулевой символ окончания строки '\0' не копируется. Функция возвращает последний записанный символ.

При ошибке возвращается значение EOF.

FREAD

```
#include <stdio.h>

uint fread(buffer, size, count, FILE *fp)
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Функция читает объекты размера size по счетчику count из входного файла fp и располагает их в буфере buffer. Возвращает число реально прочитанных объектов.

Примечание. При чтении из stdin, код 0x0D заменяется на код 0x0A (прогон бумаги) и нет возможности редактировать вводимые данные. Нажатие на комбинацию клавиш Ctrl+Z или Ctrl+C воспринимается как конец ввода данных.

Для ввода строки символов из stdin лучше использовать функцию gets, допускающую редактирование вводимых данных.

FREE

```
#include <malloc.h> или <stdlib.h>

void free(char *ptr)
```

Описание.

Функция освобождает блок памяти. Аргумент ptr указывает на ранее захваченный блок памяти посредством вызова malloc, calloc, realloc.

Возвращаемого значения нет.

FREOPEN

```
#include <stdio.h>

FILE *freopen(pathname, mode, FILE *fp)
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Функция закрывает текущий файл, связанный с дескриптором (FILE-структурой) fp и переназначает этот дескриптор на новый файл, определяемый path-именем.

Ошибки при закрытии файла игнорируются. Эта функция обычно применяется для переадресации предоткрытых потоков stdin, stdout, stderr, stdaux и stdprn в файлы, определяемые пользователем. Но поскольку ДОС Estex не поддерживает эти потоки, то при попытке переадресации stdin, stdout, stderr, stdaux и stdprn, будет возвращен NULL. Новый файл открывается в режиме mode.

Функция возвращает указатель на fp вновь открытого файла. При ошибке, функция возвращает значение NULL.

FSCANF

```
#include <stdio.h>

int fscanf(FILE *fp, char *format, arg1, arg2, ...)
```

Описание.

Функция читает данные с преобразованием формата из файла *fp, в переменные, определяемые указателями аргументов args. Каждый аргумент должен указывать на значение с типом, который соответствует типу, заданному в строке формата. Функция использует буфер в области стека, размером 128 байт.

Функция возвращает число полей, разделенных спец-символами. При достижении конца файла, возвращается EOF.

FSEEK

```
#include <stdio.h>

struct fpoint *fseek(FILE *fp, high, low, type)
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Функция устанавливает указатель в файле на новое место расположения, определяемое аргументами high и low, соответственно старшая и младшая часть 32-х битного значения перемещения. Тип перемещения определяется аргументом type.

Следующая операция в файле выполняется на новом месте расположения указателя.

Аргумент type может быть одной из следующих констант, определенных в <stdio.h>

SEEK_SET	от начала файла
SEEK_CUR	от текущей позиции указателя в файле
SEEK_END	от конца файла

Функция возвращает структуру f_point, определенную в "types.h".

При ошибке возвращается -1 и errno содержит номер ошибки. Установка указателя за пределами файла, ошибки не вызывает. Для устройств, не приспособленных к поиску (таких как терминалы и принтеры), возвращается значение -1.

FSETPOS

```
#include <stdio.h>

char fsetpos(FILE *fp, fpos_t *position)
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Функция fsetpos устанавливает положение указателя в файле равным значению, находящемуся в position, которое получается при вызове функции fgetpos для данного файла. Функция очищает индикатор конца файла. Значение position хранится во внутреннем формате и оно должно использоваться только с функциями fgetpos и fsetpos.

Функция возвращает 0. При ошибке, возвращается -1.

FTELL

```
#include <stdio.h>

struct fpoint *ftell(FILE *fp)
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Функция ftell получает текущую позицию указателя в файле. Позиция выражается как число байтов от начала файла. Функция возвращает структуру f_point, определенную в <types.h>.

При ошибке возвращается -1 и errno содержит номер ошибки. Для устройств, не приспособленных к поиску (таких как терминалы и принтеры), возвращается значение -1.

FWRITE

```
#include <stdio.h>

uint fwrite(buffer, size, count, FILE *fp)
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Функция записывает объекты размера size по счетчику count из буфера buffer в выходной файл fp.

Возвращает число реально записанных объектов.

GETC

```
#include <stdio.h>

char getc(FILE *fp)
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Функция читает текущий символ из потока fp и ставит указатель (если он есть) на следующий символ. Возвращается считанный символ.

В случае возникновения ошибки или достижения конца файла, возвращается значение EOF. Однако, значение EOF может быть воспринято как целая величина, тогда нужно использовать функции ferror и feof для проверки ошибки или конца файла.

GETCH

```
#include <conio.h>

char getch()
```

Описание.

Функция читает символ с клавиатуры без отображения на экране.

GETCHE

```
#include <conio.h>

char getche()
```

Описание.

Функция читает символ с клавиатуры с отображением на экране. При нажатии на Ctrl+Z или Ctrl+C, возвращает значение -1.

GETCURSMOUSE

```
#include <mouse.h>

MSCURS *getCursMouse(char *buffer)
```

Описание.

Функция получает изображение курсора из драйвера мыши и сохраняет его в буфере buffer.

Функция возвращает указатель на заполненную структуру MSCURS, определенную в <mouse.h>.

При ошибке, возвращается значение -1 (0FFFFh) и errno содержит номер ошибки.

GETSENSMOUSE

```
#include <mouse.h>
```

```
MSSENS *getSensMouse()
```

Описание.

Функция возвращает значения чувствительности перемещения мыши по вертикали и горизонтали.

Функция возвращает указатель на заполненную структуру MSSENS, определенную

в <mouse.h>.

При ошибке, возвращается значение -1 (0FFFFh) и errno содержит номер ошибки.

GETGSTATMOUSE

```
#include <mouse.h>
MSGSTAT *getGStatMouse()
```

Описание.

Функция возвращает текущие графические координаты курсора мыши и состояние ее кнопок.

Функция возвращает указатель на структуру MSGSTAT, определенную в <mouse.h>.

При ошибке, возвращается значение -1 (0FFFFh) и errno содержит номер ошибки.

GETSTATMOUSE

```
#include <mouse.h>
MSSTAT *getStatMouse()
```

Описание.

Функция возвращает текущие текстовые координаты курсора мыши и состояние ее кнопок.

Функция возвращает указатель на структуру MSSTAT, определенную в <mouse.h>.

При ошибке, возвращается значение -1 (0FFFFh) и errno содержит номер ошибки.

GETCWD

```
#include <direct.h>
char *getcwd(char *pathbuff, int len)
```

Описание.

Функция получает полное (с буквой диска) path-имя текущего каталога и сохраняет его в буфере pathbuff. В конце path-имени текущего каталога стоит символ "\" с заключительным нулем в конце строки. Аргумент len определяет максимальную длину для path-имени, которая должна быть достаточной, чтобы вместить информацию вместе с заключительным нулем. Максимальная длина path-имени в ДОС равна 256 байтов. Если длина path-имени превышает len, то path-имя обрезается на длину len, при этом символ "\" в конец строки не ставится.

Если аргумент pathbuff равен NULL, то getcwd вызывает malloc для выделения буфера размером len байтов. Этот выделенный буфер позже может быть освобожден посредством функции free.

Функция возвращает указатель на pathbuff. При ошибке возвращается NULL и errno содержит номер ошибки.

GETDATE

```
#include <dos.h>

void getdate(struct date *date)
```

Описание.

Функция возвращает системную дату в заполненной структуре date. Структура date описана в <dos.h>.

GETDISK

```
#include <dos.h>

char getdisk()
```

Описание.

Функция возвращает номер текущего диска. Номер диска соответствует: 0=A, 1=B,... и т.д.

GETENV

```
#include <stdlib.h>

char *getenv(char *varname)
```

Описание.

Функция возвращает для заданного имени переменной окружения varname указатель на значение (строку) этой переменной. Максимальная длина строки значения переменной окружения должна быть не более 255 байт.

Возвращается значение NULL, если заданная переменная на текущий момент не определена. При ошибке также возвращается NULL, но в errno будет содержаться номер ошибки.

GETS

```
#include <stdio.h>

char *gets(char buffer[255])
```

Описание.

Функция читает из стандартного потока stdin строку и запоминает ее в своем буфере buffer. Входная строка читается до тех пор, пока не встрерится код новой строки "0x0D" или число введенных символом не превысит 254. В конец строки вместо кода 0x0D заносится нуль, в отличии от функции fgets, которая сохраняет символ 0x0D.

Функция возвращает указатель на буфер. В случае ошибки, возвращаемое значение не определено.

Формат строки в буфере следующий:

+0: – байт, содержащий длину прочитанной строки, без концевого нуля.

+1: – начало строки.

GETTIME

```
#include <dos.h>

void gettime(struct time *time)
```

Описание.

Функция возвращает системное время в заполненной структуре time. Структура time описана в <dos.h>.

GOTOXY

```
#include <conio.h>

void gotoxy(int x, int y)
```

Описание.

Функция устанавливает курсор на X,Y позицию.

HARDMOUSE

```
#include <mouse.h>

char hardMouse()
```

Описание.

Обработчик аппаратного прерывания от мыши. По приходу прерывания от мыши, вызывается данная функция драйвера. В случае, когда требуется работа подпрограммы при запрещенных прерываниях, используйте данную функцию, для избежания потерь пакетов данных от мыши.

При ошибке, возвращается значение -1 и errno содержит номер ошибки.

HEX8

```
#include <stdlib.h>

void hex8(char value)
```

Описание.

Функция выводит в шестнадцатеричном формате значение value. Диапазон выводимых значений 0x00..0xFF.

Возвращаемого значения нет.

HEX16

```
#include <stdlib.h>

void hex16(uint value)
```

Описание.

Функция выводит в шестнадцатеричном формате значение value. Диапазон выводимых значений 0x0000..0xFFFF.

Возвращаемого значения нет.

HEX32

```
#include <stdlib.h>

void hex32(uint high, uint low)
```

Описание.

Функция выводит в шестнадцатеричном формате 16-ти битные значения аргументов high (ст.разряд) и low (мл.разряд) как одно число.

Возвращаемого значения нет.

HIDEMOUSE

```
#include <mouse.h>

char hideMouse(void)
```

Описание.

Функция убирает курсор мыши с экрана, но драйвер продолжает отслеживание перемещения мыши и нажатие на кнопки.

При ошибке, возвращается значение -1 и errno содержит номер ошибки.

HOME

```
#include <conio.h>

void home()
```

Описание.

Функция устанавливает курсор в левый верхний угол экрана.

INITMOUSE

```
#include <mouse.h>

char initMouse(void)
```

Описание.

Функция выполняет инициализацию драйвера мыши. Функция возвращает 0 при успешном завершении. При ошибке, возвращается значение -1.

INP

```
#include <conio.h>

char inp(uint port)
```

Описание.

Функция читает один байт из порта port. Аргумент port может быть любым беззнаковым целым числом в диапазоне от 0 до 65535.

Функция возвращает прочитанный байт из port.

INTDOS

```
#include <dos.h>

void intdos(union REGS *ri, union REGS *ro)
```

Описание.

Функция intdos осуществляет выполнение системного ДОС-вызова (с возвратом флагов условий), который определен значениями регистров, объявленных в union REGS *ri и возвращает результат системного вызова в union REGS *ro.

Тип объединения union REGS объявлен в <dos.h>. Для осуществления системного вызова intdos выполняет команду RST 10h. Перед выполнением команды функция копирует содержимое union REGS *ri в соответствующие регистры. После выполнения команды RST, функция intdos копирует значения текущих регистров в union REGS *ro. Она также копирует возвращаемый флаг условий системы в поле flag, размещенное в union REGS *ro. Если это поле является ненулевым, флаг указывает на условие ошибки.

union REGS *ri – значения регистров при вызове

union REGS *ro – значения регистров при возврате

ISALNUM

#include <ctype.h>

BOOL isalnum(char ch)

Описание.

Функция проверяет символ ch на принадлежность к алфавитно-цифровому типу (A..Z, a..z, 0..9).

ISALPHA

#include <ctype.h>

BOOL isalpha(char ch)

Описание.

Функция проверяет символ ch на принадлежность к буквам (A..Z, a..z).

ISASCII

#include <ctype.h>

BOOL isascii(char ch)

Описание.

Функция проверяет символ ch на ASCII принадлежность.

ISATTY

#include <io.h>

BOOL isatty(FD fd)

Описание.

Функция низко-уровневого ввода/вывода. Функция определяет, принадлежит ли данный дескриптор стандартным потокам (терминал, консоль,...).

Функция возвращает:

true – дескриптор принадлежит терминалу, консоли.

false – дескриптор принадлежит файлу.

ISCNTL

#include <ctype.h>

BOOL iscntl(char ch)

Описание.

Функция проверяет символ ch на принадлежность к управляющим символам (0x00..0x1F или 0x7F).

ISDIGIT

```
#include <ctype.h>
BOOL isdigit(char ch)
```

Описание.

Функция проверяет символ ch на принадлежность к цифре.

ISGRAPH

```
#include <ctype.h>
BOOL isgraph(char ch)
```

Описание.

Функция проверяет символ ch на принадлежность к печатаемым символам, исключая символ пробела (0x21..0x7E).

ISLOWER

```
#include <ctype.h>
BOOL islower(char ch)
```

Описание.

Функция проверяет символ ch на принадлежность к строчному символу.

ISPRINT

```
#include <ctype.h>
BOOL isprint(char ch)
```

Описание.

Функция проверяет символ ch на принадлежность к печатаемым символам (0x20..0x7E).

ISPUNCT

```
#include <ctype.h>
BOOL ispunct(char ch)
```

Описание.

Функция проверяет символ ch на принадлежность к знакам пунктуации.

ISSPACE

```
#include <ctype.h>
BOOL isspace(char ch)
```

Описание.

Функция проверяет символ ch на принадлежность к пробельным символам (0x09, 0x0D или 0x20).

ISUPPER

```
#include <ctype.h>

BOOL isupper(char ch)
```

Описание.

Функция проверяет символ ch на принадлежность к прописному символу.

ISXDIGIT

```
#include <ctype.h>

BOOL isxdigit(char ch)
```

Описание.

Функция проверяет символ ch на принадлежность к шестнадцатеричным цифрам (0..9, A..F, a..f).

KBHIT

```
#include <conio.h>

BOOL kbhit()
```

Описание.

Функция проверяет нажатие клавиши на клавиатуре.

Возвращает:

```
true  - клавиша нажата

false - клавиша не нажата
```

LONGJMP

```
#include <setjmp.h>

void longjmp(env, retval)

      jmp_buf env;      - переменная, в которой хранится окружение

      int retval;       - значение, возвращаемое при вызове setjmp
```

Описание.

Функция longjmp восстанавливает состояние стека, ранее сохраненное в env функцией setjmp.

Функции setjmp и longjmp обеспечивают возможность выполнения нелокального (nonlocal) перехода и обычно используются для передачи управления к обработке ошибок для восстановления кода в ранее вызванной процедуре (без использования обычного вызова), для возврата условных обозначений. Вызов setjmp активизирует сохранение текущего состояния стека в env. Последующий вызов longjmp восстанавливает сохраненное состояние и возвращает управление на указатель (точку входа), непосредственно следующий за соответствующим вызовом setjmp.

Функция longjmp должна вызываться ранее функции, определяющей возврат setjmp. Если longjmp вызвана после функции, определяющей возврат setjmp, то может произойти непредсказуемое поведение программы.

Значение retval, возвращаемое longjmp, должно быть ненулевым. Если для retval задан аргумент 0, значение возврата заменяется значением 1.

Функция longjmp не возвращает никакого значения.

LSEEK

```
#include <io.h>

struct fpoint *lseek(FD fd, uint high, uint low, int type)

struct fpoint *lseek(FD fd, uint high, uint low, char type)
```

Описание.

Функция низко-уровневого ввода/вывода. Функция устанавливает указатель в файле на новое место расположения, определяемое аргументами high и low, соответственно старшая и младшая часть 32-х битного значения перемещения.

Тип перемещения определяется аргументом type. Следующая операция в файле выполняется на новом месте расположения указателя. Аргумент type может быть одной из следующих констант, определенных в <io.h>

```
SEEK_SET    от начала файла

SEEK_CUR    от текущей позиции указателя в файле

SEEK_END    от конца файла
```

Функция возвращает структуру f_point, определенную в <types.h>.

При ошибке возвращается -1 и errno содержит номер ошибки.

Установка указателя за пределами файла, ошибки не вызывает.

Для устройств, не приспособленных к поиску (таких как терминалы и принтеры), возвращается значение -1.

LTELL

```
#include <io.h>

struct fpoint *ltell(FD fd)
```

Описание.

Функция низко-уровневого ввода/вывода. Функция получает текущую позицию указателя в файле. Позиция выражается как число байтов от начала файла.

Функция возвращает структуру f_point, определенную в <types.h>.

При ошибке возвращается -1 и errno содержит номер ошибки.

Для устройств, не приспособленных к поиску (таких как терминалы и принтеры), возвращается значение -1.

MALLOC

```
#include <malloc.h> или <stdlib.h>

char *malloc(int size)
```

Описание.

Функция захватывает блок памяти размером в size байтов. Функция возвращает указатель на выделенный блок памяти. Если свободной памяти недостаточно, возвращается NULL.

MAX

```
#include <stdlib.h>

int max(int arg1, int arg2)
```

Описание.

Функция сравнивает два аргумента и возвращает значение наибольшего аргумента. Тип аргументов и тип возвращаемого значения одинаков.

MEMCHR

```
#include <string.h>

void *memchr(char *buff, ch, count)
```

Описание.

Функция ищет первые count байт в buff с целью обнаружения первого появления символа ch. Поиск продолжается до тех пор, пока не будет найден указанный символ или не будут проверены все count байты.

Функция возвращает указатель на расположение символа ch в buff.

Если символ ch не обнаружен в первых count байтах buff, возвращается NULL.

MEMCMP

```
#include <mem.h>

int memcmp(*buf1, *buf2, count)
```

Описание.

Функция производит последовательное лексиграф. сравнение первых count байтов из buf1 и buf2, и продолжает это, пока они совпадают, либо пока не выполнится заданное количество сравнений count.

Функция возвращает значение, указывающее на соотношение этих буферов:

```
< 0   buf1 < buf2

= 0   buf1 = buf2

> 0   buf1 > buf2
```

MEMCPY

```
#include <mem.h>

void *memcpy(*dest, *source, count)
```

Описание.

Функция копирует count байтов из source в dest. Если некоторые участки source и dest перекрываются, функция гарантирует, что байты из перекрываемого участка, принадлежащие source, будут скопированы перед перезаписью.

Функция возвращает указатель на dest.

MEMMOVE

```
#include <mem.h>

void *memmove(*dest, *source, count)
```

Описание.

Функция копируют count байтов из source в dest. Если некоторые области в source и dest перекрываются, функция гарантирует, что исходные source байт из перекрывающейся области будут скопированы перед переписыванием.

Функция возвращает указатель на dest.

MEMSET

```
#include <mem.h>

void *memset(*dest, ch, count)
```

Описание.

Функция заполняет первые count байт буфера dest символом ch.

Функция возвращает указатель на буфер dest.

MIN

```
#include <stdlib.h>

int min(int arg1, int arg2)
```

Описание.

Функция сравнивает два аргумента и возвращает значение наименьшего аргумента. Тип аргументов и тип возвращаемого значения одинаков.

MKDIR

```
#include <direct.h>

char mkdir(char *pathname)
```

Описание.

Функция создает новый каталог с заданным path-именем. За один вызов функции может быть создан только один новый каталог, с именем последней компоненты pathname.

Функция возвращает 0, если новый каталог успешно создан. При ошибке, возвращается -1 (0FFh) и errno содержит номер ошибки.

MOVMEM

```
#include <mem.h>

void movmem(*source, *destin, len)
```

Описание.

Функция movmem копирует len байт из source в destin. Если исходная и результирующия строка перекрываются, копирование все-равно производится корректно.

OPEN

```
#include <io.h>

uint open(char *pathname, int oflag)

uint open(char *pathname, char oflag)
```

Описание.

Функция низко-уровневого ввода/вывода. Функция open открывает файл, определяемый по path-имени и подготавливает его к последующему чтению или записи, что определяется посредством oflag.

Аргумент oflag является целым выражением, состоящим из комбинации одной или более манифестных констант, объявленных в <io.h>. Если задано больше одной константы, то они объединяются при помощи логического оператора ИЛИ "|". Значения аргумента oflag приводятся ниже:

O_RDWR Файл открыт одновременно для чтения и записи. Если задается этот флаг, может быть выбран либо флаг O_RDONLY, либо O_WRONLY.

O_RDONLY Файл открыт только для чтения. Если задается этот флаг, может быть выбран либо флаг O_RDWR, либо O_WRONLY.

O_WRONLY Файл открыт только для записи. Если задан этот флаг, может быть задан также либо флаг O_RDONLY, либо ORDWR.

O_TRUNC Существующий файл открыт на чтение и запись, и усечен к длине 0. Этот файл должен иметь разрешение на запись. Содержимое файла уничтожается.

O_CREAT Новый файл создан и открыт для чтения/записи, это не эффективно, если существует файл, определяемый по path-имени.

O_EXCL Возвращается значение ошибки, если существует файл, определяемый по path-имени. Применяется только вместе с O_CREATE.

O_APPEND Указатель в файле переместить на конец файла. Для режимов O_RDONLY, O_TRUNC и O_CREAT не имеет смысла.

Замечание! O_TRUNC полностью уничтожает содержимое существующего файла, поэтому нужно внимательно использовать эту константу.

Функция возвращает номер дескриптора файла, если Ok.

При ошибке, возвращает значение -1 (0FFFFh) и ячейка errno содержит номер ошибки.

OUTP

```
#include <conio.h>

char outp(uint port, char value);
```

Описание.

Функция записывает значение value в порт port. Аргумент port может быть любым беззнаковым целым числом в диапазоне от 0 до 65535. Аргумент value может быть любым целым числом в диапазоне от 0 до 255.

Функция возвращает записываемый байт.

PERROR

```
#include <stdlib.h>

void perror(char *string)
```

Описание.

Функция выводит сообщение об ошибке на stderr (экран). В этом сообщении аргумент string выводится первым, затем выводится двоеточие, системное сообщение об ошибке от последнего библиотечного вызова, вернувшего ошибку и новая строка. Если аргумент string равен NULL, то функция пропускает сообщение string, заданное пользователем и выводит только системное сообщение об ошибке, и новую строку.

Чтобы вывести правильный результат, функция perror должна быть вызвана сразу же после библиотечной функции, возвратившей ошибку. Иначе значение errno может быть перезаписано при последующих вызовах библиотечных функций.

Функция не возвращает никакого значения.

PRINTF

```
#include <stdio.h>

int printf(char *format, arg1, arg2, ...)
```

Описание.

Функция выводит данные с преобразованием формата в стандартный поток stdout из переменных, определяемых аргументами args. Каждый аргумент должен указывать на значение с типом, который соответствует типу, заданному в строке формата.

Функция возвращает число выведенных символов. При ошибке вывода, функция возвращает отрицательное значение.

PUTC

```
#include <stdio.h>

char putc(char ch, FILE *fp)
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Записывает символ ch в файл fp. Функция возвращает записанный символ.

При ошибке, возвращается значение EOF.

PUTCH

```
#include <conio.h>

void putch(char *c)
```

Описание.

Функция выводит символ прямо на экран.

PUTENV

```
#include <stdlib.h>

uint putenv(char *envstring)
```

Описание.

Функция добавляет новую переменную окружения. Аргумент envstring должен быть указателем на строку следующего формата:

```
VARNAME=VALUE
```

где: VARNAME – имя добавляемой переменной окружения

VALUE – значение этой переменной

Элементы строки должны разделяться символом "=" и строка должна оканчиваться нулем.

Функция возвращает 0, если она успешно отработала.

При ошибке, возвращается значение -1 и errno содержит номер ошибки.

PUTS

```
#include <stdio.h>
char puts(*string)
```

Описание.

Функция записывает строку string в стандартный поток вывода stdout, заменяя в выходном потоке нулевой символ окончания строки ('\0') символом новой строки ('\n').

Функция всегда возвращает последний записанный символ, которым является символ новой строки '\n'.

QSORT

```
#include <stdlib.h>
void qsort(*base, nel, size, *compare)
      void *base;       -  указатель на таблицу
      unsigned nel;     -  количество элементов таблицы
      unsigned size;    -  размер в байтах элемента таблицы
      int (*compare)();-  указатель на функцию сравнения
```

Описание.

Функция реализует быструю сортировку таблицы данных. Аргумент compare является указателем на процедуру, поставляемую пользователем.

В процессе сортировки, функция qsort может вызывать эту процедуру один или несколько раз, при каждом вызове передавая указатели на два элемента массива. Процедура compare должна сравнивать элементы, а затем возвращать одно из следующих int-значений:

```
 < 0      element1 < element2
   0      element1 = element2
 > 0      element1 > element2
```

После отработки функции qsort, таблица содержит отсортированные элементы.

В случае ошибки, возвращаемого значения нет.

RAND

```
#include <stdlib.h>
int rand()
```

Описание.

Функция возвращает псевдослучайное целое в интервале 0..65536. Перед вызовом rand, может быть вызвана функция srand для установки начальной случайной точки.

READ

```
#include <io.h>

size_t read(FD fd, void *buff, count)
```

Описание.

Функция низко-уровневого ввода/вывода. Функция читает число байт count из файла в буфер buff. Операция чтения начинается с текущей позиции указателя в файле. Функция возвращает число реально прочитанных байтов, которое однако, может быть меньше заявленного в count (например встретится конец файла). Максимальное число байтов, которое можно прочитать из файла за один раз, равно 65534; поскольку 65535 (0xFFFF) является неотличимым от -1, поэтому будет возвращена ошибка.

Функция возвращает 0, если делается попытка чтения конца файла.

При ошибке чтения, возвращается -1 и errno содержит номер ошибки.

REALLOC

```
#include <malloc.h>

char *realloc(*ptr, int size)
```

Описание.

Функция изменяет размер ранее выделенного блока памяти. Аргумент ptr указывает на начало блока. Аргумент size задает новый размер блока в байтах. Содержимое блока не изменяется. Если size равен 0, блок освобождается и возвращается NULL. Если ptr равен NULL, то отводится (по malloc) size байтов памяти и возвращается указатель на этот блок.

Аргумент ptr может указывать на блок, который должен быть освобожден прежде, чем последуют вызовы malloc, calloc, realloc.

Функция возвращает указатель на перезахваченный блок памяти.

Блок может быть передвинут, если его размеры изменены, поэтому аргумент ptr для функции realloc не обязательно должен быть таким же, как и возвращаемое значение.

Возвращается значение NULL, если памяти недостаточно для расширения блока к заданному размеру. При этом первоначальный блок остается неизменным.

REFRESHMOUSE

```
#include <mouse.h>

char refreshMouse()
```

Описание.

Функция выполняет принудительную перерисовку изображения курсора мыши на экране не дожидаясь очередного прерывания.

При ошибке, возвращается значение -1 и errno содержит номер ошибки.

REMOVE

```
#include <io.h>
```

```
char remove(char *pathname)
```

Описание.

Функция удаляет файл, определенный по pathname. Глобальные символы * и ? в имени файла не допускаются. Файлы с атрибутом "только для чтения" нельзя удалить данной функцией – предварительно нужно изменить этот атрибут.

Функция возвращает 0, если файл успешно удален. При ошибке, возвращается –1 (0FFh) и errno содержит номер ошибки.

RENAME

```
#include <io.h>

char rename(char *oldname, char *newname)
```

Описание.

Функция переименовывает файл или каталог, определенный в имени newname. Oldname должно задавать path-имя существующего файла или каталога.

Newname не определяет имя уже существующего файла или каталога. Функцию rename нельзя использовать для перемещения файла из одного каталога в другой, указав другое path-имя в аргументе newname.

Функция возвращает 0, если операция прошла успешно.

При ошибке, возвращается –1 (0FFh) и errno содержит номер ошибки.

REWIND

```
#include <stdio.h>

void rewind(FILE *fp)
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Функция перемещает указатель в файле к началу файла. Вызов функции эквивалентен вызову следующей функции:

```
fseek(FILE *fp, 0, 0, SEEK_SET);
```

с учетом того, что в FILE-структуре fp описателя файла функция rewind очищает флаг ошибки и индикатор конца файла, а fseek этого не делает.

Функция не возвращает никакого значения.

RMDIR

```
#include <direct.h>

char rmdir(char *pathname)
```

Описание.

Функция удаляет пустой каталог, определенный в pathname. Нельзя удалить текущий каталог или корневой.

Функция возвращает 0, если каталог успешно удален.

При ошибке, возвращается –1 (0FFh) и errno содержит номер ошибки.

SBRK

```
#include <malloc.h>

char *sbrk(int n)
```

Описание.

Функция выделяет блок длиной n в пределях между указателем свободной памяти и указателем стека. Значение n может быть отрицательным, тогда количество захваченной памяти уменьшается на n байтов.

При успешном выделении возвращает указатель на выделенный блок.

Если памяти недостаточно, возвращает -1 (0FFFFh).

SCANF

```
#include <stdio.h>

int scanf(char *format, arg1, arg2, ...)
```

Описание.

Функция читает данные с преобразованием формата из стандартного потока stdin в переменные, определяемые указателями аргументов args.

Каждый аргумент должен указывать на значение с типом, который соответствует типу, заданному в строке формата. Функция использует буфер в области стека, размером 128 байт.

Функция возвращает число полей, разделенных спец-символами.

SEEK

```
#include <io.h>

uint seek(FD fd, uint offset, int type)

uint seek(FD fd, uint offset, char type)
```

Описание.

Функция низко-уровневого ввода/вывода. Функция устанавливает указатель в файле на новое место расположения, которое определяется как смещение offset. Тип перемещения определяется аргументом type. Следующая операция в файле выполняется на новом месте расположения указателя. Диапазон перемещения указателя не может быть больше 65534. Если нужно переместить указатель на большее значение, необходимо использовать функцию lseek. Аргумент type может быть одной из следующих констант, определенных в <io.h>

```
SEEK_SET   от начала файла

SEEK_CUR   от текущей позиции указателя в файле

SEEK_END   от конца файла
```

Функция возвращает значение нового положения указателя. При ошибке возвращается -1 и errno содержит номер ошибки. Установка указателя за пределамифайла, ошибки не вызывает. Для устройств, не приспособленных к поиску (таких как терминалы и принтеры), возвращаемое значение -1.

SETDATE

```
#include <dos.h>
```

void setdate(struct date *date)

Описание.

Функция устанавливает новую системную дату, используя данные структуры date. Структура date описана в <dos.h>.

SETDISK

#include <dos.h>

void setdisk(char diskno)

Описание.

Функция устанавливает новый текущий диск, номер которого задан в diskno. Номер диска соответствует: 0=A, 1=B,... и т.д.

SETJMP

#include <setjmp.h>

int setjmp(env)

jmp_buf env – переменная, в которой хранится сохраняемое окружение.

Описание.

Функция setjmp сохраняет состояние стека, который может быть последовательно восстановлен использованием функции longjmp.

Функции setjmp и longjmp обеспечивают возможность выполнения нелокального (nonlocal) перехода и обычно используются для передачи управления к обработке ошибок для восстановления кода в ранее вызванной процедуре (без использования обычного вызова), для возврата условных обозначений. Вызов setjmp активизирует сохранение текущего состояния стека в env. Последующий вызов longjmp восстанавливает сохраненное состояние и возвращает управление на указатель (точку входа), непосредственно следующий за соответствующим вызовом setjmp.

После сохранения состояния стека функция setjmp возвращает значение 0.

Возвращаемого значения в случае ошибки нет.

SETCURSMOUSE

#include <mouse.h>

char setCursMouse(MSCURS *p)

Описание.

Функция загружает новое изображение курсора мыши в драйвер и задает активную точку изображения. Перед вызовом функции, должна быть заполнена структура MSCURS, определенная в <mouse.h>.

При ошибке, возвращается значение –1 и errno содержит номер ошибки.

SETSENSMOUSE

#include <mouse.h>

char setSensMouse(MSSENS *p)

Описание.

Функция устанавливает значения чувствительности перемещения мыши по вертикали и горизонтали. Перед вызовом функции, должна быть заполнена структура MSSENS, определенная в <mouse.h>.

При ошибке, возвращается значение -1 и errno содержит номер ошибки.

SETGPOSMOUSE

```
#include <mouse.h>
char setGPosMouse(int x, int y)
```

Описание.

Функция перемещает курсор мыши в заданные графические координаты.

При ошибке, возвращается значение -1 и errno содержит номер ошибки.

SETPOSMOUSE

```
#include <mouse.h>
char setPosMouse(int col, int line)
```

Описание.

Функция перемещает курсор мыши в заданные текстовые координаты.

При ошибке, возвращается значение -1 и errno содержит номер ошибки.

SETTXTMOUSE

```
#include <mouse.h>
char setTxtMouse(MSTXT *p)
```

Описание.

Функция устанавливает символ и атрибут курсора мыши для текстового режима.

При ошибке, возвращается значение -1 и errno содержит номер ошибки.

SETMEM

```
#include <mem.h>
void setmem(*addr, len, char value)
```

Описание.

Функция устанавливает первые len байт блока addr в значение value.

SETTIME

```
#include <dos.h>
void settime(struct time *time)
```

Описание.

Функция устанавливает новое системное время, используя данные структуры time. Структура time описана в <dos.h>.

SHOWMOUSE

```
#include <mouse.h>

char showMouse(void)
```

Описание.

Функция выводит на экран курсор мыши, отслеживает все перемещения курсора и перерисовывает его.

При ошибке, возвращается значение −1 и errno содержит номер ошибки.

SLEEP

```
#include <dos.h>

void sleep(uint seconds)
```

Описание.

Функция приостанавливает выполнение программы на seconds секунд.

Значение seconds может находиться в диапазоне 0..65534. После выполнения функции, прерывания остаются включенными.

SPRINTF

```
#include <stdio.h>

int sprintf(char buffer, char *format, arg1, arg2, ...)
```

Описание.

Функция преобразует формат и выводит наборы символов и значений в buffer. В конец выводимой строки записывается нуль. Каждый аргумент должен указывать на значение с типом, который соответствует типу, заданному в строке формата.

Функция возвращает количество символов, записанных в buffer.

SRAND

```
#include <stdlib.h>

void srand(seed)
```

Описание.

Функция srand устанавливает начальную точку для генерации множества псевдослучайных целых. В качестве аргумента seed для переустановки генератора используется 1. Любое другое значение seed устанавливает генератор в начальную случайную точку. Возвращаемого значения нет.

SSCANF

```
#include <stdio.h>

int sscanf(char *buffer, char *format, arg1, arg2, ...)
```

Описание.

Функция читает данные с преобразованием формата из buffer, в переменные, определяемые указателями аргументов args. Каждый аргумент должен указывать на значение с типом, который соответствует типу, заданному в строке формата.

Функция возвращает число полей, разделенных спец-символами.

STRCAT

```
#include <string.h>
char *strcat(*s1, *s2)
```

Описание.

Функция добавляет s2 к s1, завершая строку s1 нулевым символом.

Функция возвращает указатель на s1.

STRCHR

```
#include <string.h>
char *strchr(*string, ch)
```

Описание.

Функция находит первое вхождение ch в string. Символ ch может быть нулевым символом. Нулевой символ окончания string также включается в поиск.

Функция возвращает указатель на первое вхождение ch в string.

Возвращается NULL, если символ не найден.

STRCMP

```
#include <string.h>
int strcmp(*s1, *s2)
```

Описание.

Функция производит последовательное регистро-зависимое сравнение символов строк s1 и s2, и продолжает это, пока они совпадают, либо пока одна из строк не закончится.

Функция возвращает значение, указывающее на соотношение этих строк:

```
< 0   s1 < s2
= 0   s1 = s2
> 0   s1 > s2
```

STRCPY

```
#include <string.h>
char *strcpy(*s1, *s2)
```

Описание.

Функция копирует s2 в s1 (включая 0-й символ окончания строки).

Функция возвращает указатель на s1.

STRCSPN

```
#include <string.h>
size_t strcspn(*s1, *s2)
```

Описание.

Функция находит первую часть строки в s1, не входящую в строку s2.

Возвращает длину части строки s1, состоящей из символов, не найденных в строке s2. Если ни один из символов строки s1 не найден в s2, возвращается общая длина строки s1 без завершающего нуля.

STRERR

```
#include <string.h>

char *strerr(char errno)
```

Описание.

Функция strerr (аналогична strerror) преобразует errno в строку системного сообщения об ошибке и возвращает указатель на эту строку.

Функция, в отличие от perror, не выводит никаких сообщений. Для вывода сообщения требуется вызвать функцию вывода (например cprintf).

Номер ошибки хранится в переменной errno, которая объявлена на внешнем уровне и используется как индекс в массиве системных сообщений.

Системные сообщения об ошибках доступны через переменную _errlst_, являющуюся массивом системных сообщений об ошибках.

Чтобы вернуть правильный результат, функция strerr должна быть вызвана сразу после библиотечной функции, возвратившей ошибку. Иначе, значение errno может быть перезаписано при последующих вызовах библиотечных функций. Для любого значения errno, не используемого в ДОС, функция strerr возвращает указатель на пустую строку.

STRLEN

```
#include <string.h>

size_t strlen(*string)
```

Описание.

Функция возвращает в байтах длину строки string, причем нулевой символ окончания строки, в размер строки не входит.

STRLWR

```
#include <string.h>

char *strlwr(*string)
```

Описание.

Функция преобразовывает любые буквы верхнего регистра строки string, заканчивающейся нулем, в буквы нижнего регистра. Другие символы не рассматриваются. Функция возвращает указатель на строку string.

STRNCAT

```
#include <string.h>

char *strncat(*s1, *s2, count)
```

Описание.

Функция добавляет count символов строки s2 в конец строки s1, завершая строку s1 нулевым символом. В s1 должно быть достаточно места для размещения результирующей строки. Функция возвращает указатель на s1.

STRNCMP

```
#include <string.h>

int strncmp(*s1, *s2, uint count)
```

Описание.

Функция производит последовательное регистро-зависимое сравнение символов строк s1 и s2. Количество сравниваемых символов задается в count.

Если одна из строк заканчивается, сравнение прекращается.

Функция возвращает значение, указывающее на соотношение этих строк:

```
< 0   s1 < s2

= 0   s1 = s2

> 0   s1 > s2
```

STRNCPY

```
#include <string.h>

char *strncpy(*s1, *s2, count)
```

Описание.

Функция копирует точно count символов строки s2 в строку s1. Если count меньше длины s2, нулевой символ окончания строки не добавляется к строке s1. Если count больше длины s2, то s1 заполняется нулевыми символами, вплоть до достижения длины count. Функция возвращает указатель на s1.

STRPBRK

```
#include <string.h>

char *strpbrk(*s1, *s2)
```

Описание.

Функция находит первое вхождение в строке s1 любого символа из строки s2. Нулевой символ окончания при поиске не рассматривается.

Функция возвращает указатель на первое вхождение символа из s2 в строке s1. При отсутствии общих символов в s1 и s2, возвращается NULL.

STRRCHR

```
#include <string.h>

char *strrchr(*string, ch)
```

Описание.

Функция находит последнее вхождение символа ch в строке string. Нулевой символ окончания строки string также включается в поиск.

Функция возвращает указатель на последнее вхождение символа ch в string. Если заданный символ не найден, возвращается NULL.

STRSPN

```
#include <string.h>

size_t strspn(*s1, *s2)
```

Описание.

Функция возвращает длину совпадающей части строки s1, которая полностью состоит из символов, найденных в строке s2. Нулевой символ окончания строки s2 не рассматривается. Если строка s1 начинается с сим вола, не входящего в строку s2, возвращается 0.

STRSTR

```
#include <string.h>

char *strstr(*s1, *s2)
```

Описание.

Функция возвращает указатель на первое вхождение строки s2 в строку s1. Функция возвращает NULL, если s2 не была найдена. Если длина строки s2 равна 0, возвращается s1.

STRTOK

```
#include <string.h>

char *strtok(*s1, *s2)
```

Описание.

Функция находит в строке s1 токен, отделенный одним (или более) ограничителем из множества символов-ограничителей строки s2. Найденный токен в строке s1 оканчивается нулем. Для того, чтобы прочитать следующий токен из s1, функция strtok должна вызываться с NULL-значением для аргумента s1. Аргумент NULL s1 вызывает strtok на поиск следующего знака в предыдущей строке символов.

Множество ограничителей в строк s2, от вызова к вызову, может быть различным, так что s2 может принимать любые значения. При каждом вызове, функция возвращает указатель на найденный токен строки s1, каждый раз продвигаясь вперед по строке s1.

Функция возвращает указатель NULL, когда больше нет токенов в строке s1.

STRTOL

```
#include <stdlib.h>

int strtol(char *s, char **eptr, int base)

      char *s          указатель на строку

      char **eptr      указатель на конец просмотра

      int base         используемое основание числа
```

Описание.

Функция преобразует символьную строку s в long-значение, эквивалентное числу с заданным base основанием. Функция прекращает чтение строки, как только встретится

символ, который не может быть частью числа – это может быть нулевой символ '\0' в конце строки или первый цифровой символ, который больше или равен base. Если eptr не равен NULL, *eptr указывает на тот символ, на котором останавливается просмотр.

Функция strtol предполагает, что s указывает на строку следующей структуры:

```
[whitespace][sign][0][x][digits]
```

Если base лежит между 2 и 36, это значение используется, как основание системы счисления данного числа. Если base равно 0, начальные символы строки, на которые указывает s, используются для определения основания. Если первый символ равен 0 и второй – одна из цифр от 1 до 7, то строка интерпретируется как восьмеричное целое. Если первый символ равен 0, а второй 'x' или 'X', тогда строка понимается как шестнадцатеричное целое. Если первый символ принадлежит к цифрам 1..9, то строка понимается как десятичное целое.

Функция strtol возвращает значение преобразованной строки, если не было переполнения или потери результата.

STRUPR

```
#include <string.h>

char *strupr(*string)
```

Описание.

Функция преобразовывает любые буквы нижнего регистра строки string, заканчивающейся нулем, в буквы верхнего регистра. Другие символы не рассматриваются. Функция возвращает указатель на строку string.

SYSENV

```
#include <stdlib.h>

char *sysenv(char *sysenv)
```

Описание.

Функция получает системное окружение. Аргумент sysenv должен быть указателем на буфер, предназначенный для данных системного окружения. Размер буфера должен быть достаточным, чтобы вместить все данные системного окружения. Формат данных в буфере следующий:

- строка каждой переменной окружения заканчивается на нуль.
- в конце строки последней переменной окружения, стоит дополнительный нуль, показывающий, что дальше переменных нет.

Пример (в асмовом варианте):

```
sysenv: db    "PATH=c:\fn;c:\zx",0
        db    "SOLID=H",0
        db    0
```

Функция возвращает указатель на буфер sysenv. При ошибке, возвращается значение –1 и errno содержит номер ошибки.

TELL

```
#include <io.h>

uint tell(FD fd)
```

Описание.

Функция низко-уровневого ввода/вывода. Функция возвращает текущую позицию указателя в файле. Позиция выражается как число байтов от начала файла. Значение возвращаемого указателя не может быть больше 65534. Если нужно получить большее значение, необходимо использовать функцию ltell.

При ошибке возвращается -1 и errno содержит номер ошибки.

Для устройств, не приспособленных к поиску (таких как терминалы и принтеры), возвращаемое значение -1.

TOLOWER

```
#include <ctype.h>

char tolower(char ch)
```

Описание.

Функция преобразует прописной символ ch в строчный.

TOUPPER

```
#include <ctype.h>

char toupper(char ch)
```

Описание.

Функция преобразует строчный символ ch в прописной.

UNGETC

```
#include <stdio.h>

char ungetc(char ch, FILE *fp)
```

Описание.

Функция относится к функциям высоко-уровневого ввода/вывода. Функция возвращает символ ch в файл. Последующая операция чтения из файла начинается с символа ch.

При удачном завершении, функция возвращает ch. При ошибке возвращается EOF, говорящее о неудачном возвращении заданного символа.

UNLINK

```
#include <io.h>

char unlink(char *pathname)
```

Описание.

Функция удаляет файл, определенный по pathname. Глобальные символы * и ? в имени файла не допускаются. Файлы с атрибутом "только для чтения" нельзя удалить данной функцией - предварительно нужно изменить этот атрибут.

Функция возвращает 0, если файл успешно удален. При ошибке, возвращается -1 (0FFh) и errno содержит номер ошибки.

VMODEMOUSE

```
#include <mouse.h>

char vmodeMouse(char video_mode)
```

Описание.

Функция информирует драйвер мыши о смене режима экрана. Номера режимов соответствуют функции установки режима экрана.

При ошибке, возвращается значение -1 и errno содержит номер ошибки.

WRITE

```
#include <io.h>

size_t write(FD fd, void *buff, count)
```

Описание.

Функция низко-уровневого ввода/вывода. Функция записывает число байт count из буфера buff в файл. Операции записи начинаются с текущей позиции указателя в файле. Если файл открыт для добавления, операции выполняются в конец файла. После осуществления операций записи, указатель в файле увеличивается на количество реально записанных байтов. Макс. число записываемых байтов в файл равно 65534; поскольку число 65535 является неотличимым от -1, то возвращается ошибка.

Функция возвращает число реально записанных байт. При ошибке записи возвращается значение -1 и в errno устанавливается номер ошибки.

XLIMMOUSE

```
#include <mouse.h>

char xLimMouse(uint minx, uint maxx)
```

Описание.

Задание горизонтальных границ перемещения курсора. Функция задает левую минимальную и правую максимальную графические координаты, в которых может перемещаться курсор мыши по горизонтали.

При ошибке, возвращается значение -1 и errno содержит номер ошибки.

YLIMMOUSE

```
#include <mouse.h>

char yLimMouse(int miny, int maxy)
```

Описание.

Задание вертикальных границ перемещения курсора. Функция задает верхнюю минимальную и нижнюю максимальную графические координаты, в которых может перемещаться курсор мыши по вертикали.

При ошибке, возвращается значение -1 и errno содержит номер ошибки.

Принцип раздельной компиляции

При разработке больших программ исходный текст разделен на несколько файлов.

После того, как они отдельно друг от друга откомпилированы, их нужно скомпоновать в единую программу. Здесь описан метод раздельной компиляции и отмечены основные моменты, на которые следует обращать внимание при разработке исходного текста.

Замечания к исходной программе

Имеются две серьезные проблемы раздельной компиляции и сборки больших программ:

- ссылки на функции из других файлов
- возможность получения общих данных

Ниже дана попытка освещения этих вопросов с параллельным показом на примерах. Рассмотрим следующую программу.

```
#include <stdio.h>
int  a[10];
char b[4];
char main()
{
        char sub1(),sub2();
        sub1();
        sub2();
        printf("%d,%d,%d\n",a[0],a[3],a[9]);
        printf("%c,%c,%c\n",b[0],b[1],b[2]);
}
char sub1()
{
        a[0]=1;
        a[3]=2;
        a[9]=3;
}
char sub2()
{
        b[0]='a';
        b[1]='b';
        b[2]='c';
}
```

Текст данной программы размещен в трех файлах с именами PROG1, PROG2 и PROG3 – соответственно содержащих главную функцию и функции 1 и 2.

Точнее, в каждом из этих файлов содержатся:

PROG1.C:

```
        #include <stdio.h>
        int a[10];
        char b[4];
        char main()
        {
                :
                :
        }
```

PROG2.C:

```
        extern int a[10];
        char sub1()
        {
                :
                :
        }
```

PROG3.C:

```
        extern char b[4];
        char sub2()
```

```
{
        ⋮
        ⋮
}
```

Функция main вызывает функции sub1 и sub2, поэтому их определения необходимы. Так как функции sub1 и sub2 в свою очередь никакие другие функции не вызывают – никаких других описаний не требуется. Использование массивов "a" и "b" также является проблемой. На идентификатор "a" ссылаются программа main и sub1. На массив "b" ссылаются функции main и sub2. В любом из этих вариантов между файлами должна быть установлена связь.

Для этой цели используется описатель "extern" (внешний). В первом из файлов – PROG1.C, этот описатель отсутствует, однако в двух других файлах этот описатель введен. Этот факт отражает то обстоятельство, что массивы "a" и "b" содержатся в файле PROG1.C, а в других файлах этих массивов нет. Если в двух или большем числе файлов будут стоять определения без описателей "extern", то при сборке единой программы появятся сообщения о дублировании имен массивов. С другой стороны, при добавлении описателя "extern" во все определения, они могут диагностироваться как неопределенные.

Следовательно, в случае организации общих данных для нескольких файлов необходимо дать определение без описателя "extern" только в одном из файлов с текстом исходной программы. Во все другие файлы нужно поставить "extern".

Процедура компиляции и сборки программы

Для описанных выше текстовых файлов с исходной программой последовательность команд компиляции и сборки модулей выглядит следующим образом:

```
cc1 prog1
cc1 prog2
cc1 prog3
cc2 prog1
cc2 prog2
cc2 prog3
as prog1
as prog2
as prog3
ld prog=prog1,prog2,prog3,clib/l/gXMAIN
```

В том случае, когда все части программы (все файлы) уже откомпилированы и требуется лишь перекомпилировать одни из них (например файл PROG2.C), следует выполнить приведенную ниже группу команд:

```
cc1 prog2
cc2 prog2
as prog2
ld prog=prog1,prog2,prog3,clib/l/gXMAIN
```

Создание библиотек

Известны два способа создания библиотеки. Оба они будут описаны в следующих параграфах. Первый из них, метод "a" – быстрый и легкий по исполнению, однако к собираемым пользовательским программам прилинковываются в том числе и такие функции, которые не обязательно будут использоваться.

В отличие от первого, способ "b" реализуется сложнее и требует большего времени, но получаемые в соответствии с ним пользовательские программы будут содержать в себе только те функции, которые заведомо применяются в программе.

Стандартная библиотека компилятора "clib.irl" создается именно по способу "b".

1. Простой способ создания библиотеки

Рассмотрим пример двух функций "power" и "log2":

```
int power(x,e)
int x,e;
{
     int y;
     for(y=1; e>0; e--)
           y *= x;
     return(y);
}


int log2(x)
int x;
{
     int y;
     for(y=0; x>1; x>>=1)
           y++;
     return(y);
}
```

Если эти функции помещаются в файл с именем XLIB.C, последовательность создания библиотеки выглядит следующим образом:

```
cc1 xlib
cc2 xlib
as xlib
```

При выполнении этой последовательности будет создана библиотека с именем XLIB.REL. Ниже приведена команда сборки программы, которая вызывает функции из этой библиотеки:

```
ld prog,xlib,clib/l/gXMAIN
```

В этом случае нет необходимости писать ключ /l после имени библиотеки XLIB, т.к. даже если его написать, тем не менее все функции, входящие в библиотеку XLIB, будут скомпонованы с программой "prog".

Здесь будет описан способ включения в библиотеку программ, написанных на Ассемблере. В качестве примеров взяты функции hex, ror и rol, помещенные в файл ALIB.ASM:

```
      public hex_, ror_, rol_
      cseg
hex_:             ; convert 0..F -> '0'..'F'
      and 0Fh
      add a,90h
      daa
```

```
      adc a,40h
      daa
      ret

ror_:             ; ror(c,n) n bit rotate right char
      inc e
      rorl: dec e
      ret z
      rrca
      jp rorl

rol_:             ; rol(c,n) n bit rotate left char
      inc e
      roll: dec e
      ret z
      rlca
      jp roll
```

Включение этих функций в библиотеку производится путем выполнения следующей команды:

```
as alib.asm
```

Следующая команда вызывает линковку программы, как и в случае Си-программы:

```
ld prog,alib,clib/l/gXMAIN
```

Как видно из приведенных выше примеров, самый простой способ формирования библиотеки - включение в нее тех программ, из которых компонуется программа. Однако этот способ не совершенен, так как вызывает подключение всей библиотеки.

В следующем параграфе будет описан способ создания библиотеки, при котором программа компонуется только из тех функций, к которым предполагаются обращения.

2. Создание полной библиотеки

2.1. Включение Си-программ в библиотеку

Здесь будет приведен тот же пример, что и выше - включение файла XLIB.C, содержащего тексты функций POWER и LOG2 в библиотеку.

Вначале нужно "разделить" файл XLIB.C на составляющие его функции, записав их в отдельные файлы POWER.C и LOG2.C. Далее необходимо выполнить следующие команды:

```
cc1 power
cc1 log2
cc2 power
cc2 log2
as power
```

```
as log2
ol a xlib.irl power log2
```

При выполнении этой последовательности команд, происходит следующее:

1. Си-тексты функций транслируются в ассемблерные листинги.
2. В результате ассемблирования получаем модули с расширением ".rel".
3. С помощью библиотекаря OL из двух модулей с расширением ".rel" создается библиотека XLIB.IRL.

После создания библиотеки XLIB.IRL, файлы POWER.REL и LOG2.REL уже не нужны и их можно удалить.

Применение библиотеки XLIB.IRL определяется следующей командой:

```
ld prog,xlib/l,clib/l/gXMAIN
```

После имени библиотеки XLIB нужен ключ /l как указание линкеру подключить к программе только необходимые ей функции.

При сборке программ, состоящих из большого числа файлов (функций), имеет значение порядок расположения функций. Ниже приведены два примера: один из них правильный, а другой неправильный.

Правильный пример:

```
char islower(c)
char c;
{
   return('a'<=c && c<='z');
}

char toupper(c)
char c;
{
   return(islower(c) ? c-'a'-'A' : c);
}
```

В данном примере в библиотеку включается файл X.C, состоящий из двух функций. В этом случае функция "islower" расположена в файле X.REL перед функцией "toupper". Поэтому при редактировании связей по ключу /l линкер не выдаст диагностику об ошибке типа отсутствия определения.

Но если написать вначале текст функции "toupper", возникает следующий неправильный пример:

```
char toupper(c)
char c;
{
   char islower();
```

```
    return(islower(c) ? c-'a'-'A' : c);
}

char islower(c)
char c;
{
    return('a'<=c && c<='z');
}
```

В результате расположение модулей в библиотечном файле будет неправильным. Подводя итоги сказанному, можно резюмировать, что для правильного порядка расположения модулей в библиотеке нужно в исходной программе вначале писать вызываемую функцию, а после нее – вызывающую.

2.2. Создание библиотеки модулей на Ассемблере

По сравнению с описанием включения в библиотеку, которое было дано в п.1, здесь, файл ALIB.ASM следует предварительно разделить на следующие три файла "hex.asm", "ror.asm" и "rol.asm":

```
HEX.ASM:
            cseg
hex_::      and 0Fh
            add a,90h
            daa
            adc a,40h
            daa
            ret
ROR.ASM:
            cseg

ror_::      inc e
rorl:       dec e
            ret z
            rrca
            jp rorl
ROL.ASM:
            cseg

rol_::      inc e
roll:       dec e
            ret z
            rlca
            jp roll
```

После того, как каждый из них будет ассемблирован и получен файл с расши-

рением ".rel", вызывается библиотекарь OL для создания библиотеки:

```
as hex
as ror
as rol
ol a alib.irl hex ror rol
```

Применение библиотеки ALIB.IRL определяется следующей командой:

```
ld prog,alib/l,clib/l/gXMAIN
```

При сборке программы, точно таким же образом, как и для сборки Си-программы из модулей, должна соблюдаться последовательность записи исходного текста, а именно: вызываемая функция должна быть записана прежде, а вызывающая ее, вслед за ней.

Линкер

Линкер LD предназначен для сборки готовых программ или dll-библиотек из отдельных объектных модулей. В качестве объектных модулей могут выступать файлы rel-формата или библиотеки irl-формата (создаваемые библиотекарем ol.exe). Описание "rel", "irl" форматов смотрите в файлах "rel.rus" и "irl.rus".

Строка вызова линкера имеет следующий формат:

```
LD [/keys] [outfile=] infile[/keys] [,infile[/keys] ...] [@ list]
```

где:

```
keys        - опции линкера
outfile           - имя выходного файла программы
infile      - файл "rel" или "irl" формата
@ list      - файл подстановок
```

Пример вызова линкера:

```
LD hello.rel
```

После выполнения данной команды получится файл "hello.exe".

При своей работе, линкер может создавать на диске временные файлы. Поэтому для ускорения работы линкера рекомендуется использовать его на электронном диске.

Опции линкера:

```
A                 использовать диск для сохранения симв. таблицы
D##               адрес расположения DATA сегмента
E                 не вставлять EXE заголовок
Glabel            задать метку 'label' в качестве стартовой
L                 поиск библиотечного файла (опция после имени файла)
P##               адрес расположения CODE сегмента
Q                 вывести метки вида ?labels на экран
R##               установить адрес загрузки (адрес ORG-а программы)
S                 вывести список меток на экран
T[type:ver]       создать DLL библиотеку. Type - внутр. описание библы;
                  ver - версия библы. Символ ":" разделяет параметры.
U                 заменить все "@" на "_" в именах меток
Y                 исключить DATA сегмент из выходного кода
X                 не создавать SYM файл
```

Создание DLL-библиотек

Для создания динамических библиотек служит опция "t". Обратите внимание, что эта опция отменяет вставку exe-заголовка.

Отслеживается превышение максимального размера dll-библиотеки в 16 kB. В этом случае линкер выдает предупреждающее сообщение, но линковка библиотеки не прерывается.

Создаваемые dll-библиотеки имеют сигнатуру заголовка "L1". Библиотека этого формата отличается от формата "L0" только тем, что таблица перемещений начинается сразу за концом "своего" кода.

Заголовок dll-библиотеки создается и прилинковывается автоматически, поэтому не надо резервировать для него место при написании ассемблерных текстов.

Более подробную информацию по dll-библиотекам смотрите в документации менеджера dll-библиотек.

Пример строки вызова линкера:

```
LD /TSymple library:0001 test.rel
```

где:

"Symple library"	–	Строка внутреннего описания библиотеки. Строка описания может содержать пробелы. Длина описания не должна превышать 16 символов, иначе линкер выдаст ошибку.
"0001"	–	Внутренний номер версии библиотеки. Может содержать символы 0..9 и A..F (a..f).
"test.rel"	–	объектный файл, служащий для создания библиотеки.

Можно не указывать один из параметров (или все) опции, но символ ":" разделителя параметров должен стоять всегда. В случае отсутствия описания — в заголовке библиотеки будет стоять пустая строка. В случае отсутствия номера версии – в заголовке будет стоять нулевой номер. Примеры вызова опции:

```
LD /Tsymple library: test.rel
LD /tМоя библиотека: test
LD /T:010B test.rel
LD /t:7 test
LD /t: test.rel
```

Обратите внимание, что строка внутреннего описания библиотеки начинается сразу же за буквой "t" опции. Например, если написать

```
LD /T Example:0001 test.rel
```

то строка описания в заголовке библиотеки будет иметь вид " Example", т.е. начинаться с пробела.

Файл подстановок

Файл подстановок служит для некоторой "автоматизации" процесса линковки программ. Можно один раз создать файл подстановки и линковать программу, не набирая каждый раз (возможно длинную) строку параметров линкера. Файл подстановок может содержать всего одну строку агрументов или отдельные аргументы могут начинаться с новой строки. В конце файла подстановок должна стоять пустая строка. Размер файла подстановок ограничен до 256 байт и файл может иметь любое имя.

Пример вызова линкера с файлами подстановок:

```
LD @ comp.txt
```

или

```
LD @list
```

Примеры файла подстановок.

Пример 1

```
test=prog,clib.irl/l/gxmain/x
```

Пример 2

```
test=prog,
clib.irl/l
/gxmain
/x
```

В данных примерах собирается программа с заданным именем "test". При линковке используется исходный объектный файл "prog.rel", библиотека "clib.irl" и управление в программе передается на метку "xmain" (это необходимо делать для всех программ, линкуемых с Си-библиотекой "clib.irl"). Опция "x" запрещает создание sym-файла.

Библиотекарь

Библиотекарь OL служит для работы с библиотеками IRL-формата. Информацию по этому формату можно прочитать в файле "irl.rus". Строка вызова библиотекаря имеет следующий формат:

```
OL <опция> <имя_библиотеки> [<имя_модуля1> <имя_модуля2> ...]
```

Например:

```
OL a mylib.irl module1 module2 module3
```

В данном примере в библиотеку MYLIB.IRL добавляются три новых модуля (rel-файлы). Если файла MYLIB.IRL не существует, то создается новая библиотека с таким именем, содержащая три указанных модуля.

Опции библиотекаря:

A	–	Добавить модуль(и) в библиотеку. Если указанной библиотеки нет, она создается.
D	–	Вывод дампа содержимого библиотеки на экран.
E	–	Извлечь модуль(и) из библиотеки и сохранить его в файле. В именах модулей можно использовать глобальные символы "*" и "?".
L	–	Вывод списка модулей библиотеки на экран.
R	–	Переиндексация библиотеки.
T	–	Проверка всех модулей библиотеки на неопределенные внешние метки.

IRL формат

Последняя редакция: 7.05.2004

IRL-файл представляет собой индексированный REL-файл. Благодаря индексированию улучшается работа линкера. В начале IRL-файла находятся 3 байта:

```
db    старший          ;+0  (File offset >> 14) & 0x7F
db    средний          ;+1  (File offset >>  7) & 0x7F
db    младший          ;+2  (File offset     1) & 0x7F
```

IRL-файл состоит из 3 секций: IRL-заголовок, Символьная таблица и REL-имидж.

IRL-заголовок

Массив 128 байт в начале REL-файла. Используются только первые три байта, содержащие смещение до REL-имиджа. Остальные байты установлены в ноль:

```
db    старш,средн,0     ; Смещение до REL-имиджа. Младший байт
                        ; всегда равен 0. Начало REL-имиджа вы-
                        ; равнивается на границу в 128 байтов.
ds    125               ; Не используются и установлены в 0.
```

Чтобы вычислить смещение до REL-имиджа, надо "старший" и "средний" байты умножить на 128. Например для файла clib.irl: 0011h * 128 = 0880h

```
ld    h,xx        ;+0
```

```
ld    a,xx        ;+1
;a,h,l * 128
add   a,a
ld    l,a
xor   a
srl   h
rr    l
srl   h
rr    l
rra
;hl=ст.разряд
;a=мл.разряд
```

Обратите внимание, что 7-й бит первого байта всегда 0. В не индексированном rel-файле, 7-й бит первого байта всегда 1 (первый бит записи "имя модуля").

Символьная таблица

Таблица начинается со смещения +80h от начала файла и состоит из множества символьных записей разной длины. Каждая запись имеет формат:

```
db    ст,средн,мл       ; Смещение от начала rel-имиджа до rel-модуля,
                        ; содержащего символьную строку (имя).
db    "SYMBOL"          ; Символьная строка из 1..8 символов.
db    0FEh              ; Конец строки.
```

Конец таблицы отмечается 0 в начале символьной строки:

```
db    ст,средн,0        ; Размер REL-имиджа. Это не смещение на конец
                        ; файла в REL-имидже, а показывает число байт
                        ; REL-имиджа, включая выравнивание до границы
                        ; в 128 байт.
db    0FEh              ; Конец строки.
```

Примечание!: У irl-файлов от библиотекаря "OL", данная запись имеет формат:

```
db    0,0,0
db    0FFh
```

После этой записи, таблица выравнивается (LIB использует байт 1Ah) до следующей границы в 128 байт.

REL-имидж

REL-имидж всегда начинается с границы в 128 байт. Состоит из обычных rel-данных. Чтобы преобразовать IRL-библиотеку в REL-библиотеку, надо удалить заголовок и символьную таблицу.

Формат REL-совместимых объектных файлов

Последняя редакция: 7.05.2004

Объектные файлы REL-формата фирмы Microsoft, представляют собой битовый поток. Отдельные поля в битовом потоке не выровнены на границу байта, кроме элементов, описанных ниже.

Использование битового потока для объектных файлов уменьшает их размер, сводя к минимуму число обращений к диску для чтения/записи данных.

Имеются два основных типа: Абсолютный и Перемещаемый. Первый бит служит индикатором типов. Если он равен 0, следующие 8 бит загружаются как абсолютный байт. Если первый бит равен 1, следующие 2 бита определяют 4 типа:

00b Спец-элемент (описание ниже).

01b Перемещаемый Код. Значение следующих 16 бит (2 байта) необходимо прибавить с текущему значению счетчика адреса Кода.

10b Перемещаемые Данные. Значение следующих 16 бит (2 байта) необходимо прибавить с текущему значению счетчика адреса Данных.

11b Перемещаемая Общая область (код+данные). Значение следующих 16 бит (2 байта) необходимо прибавить с текущему значению счетчика адреса Common-области.

Спец-элемент(ы) содержит:

- контр. поле из 4-х бит номеров элементов 0..15.
- не обязательное A-поле. Содержит 2 бита типа адресации cseg, dseg,common (кроме абсолютного) и 2 (4 у extended REL) байта значения.
- не обязательное B-поле. Содержит 3 бита (5 у extended REL) длины имени и самого имени идентификатора.

Общее представление спец-элемента:

```
1 00b   xxx   yy  nnnn  mmmm    zzz + имя идентификатора
              --------------    ------------------------
                  A-поле                 B-поле
```

xxx	4 бита контр. поля (0..15 номера элементов, см. ниже)
yy	2 бита типа адресации (cseg/dseg/common)
nnnn	16 бит значения (адрес)
mmmm	16 бит дополнительно, при расширенном REL-формате
zzz	3 бита длины имени (5 бит при расш. REL-формате)
...	имя идентификатора

Номера элементов контрольного поля

Номера элементов только в B-поле:

0 – имя метки, на которую имеется ссылка

1 – имя блока COMMON

2 – имя прогр. модуля (program name)

3 – request-имя файла (request library search)

4 – элемент расширения. См. примечание.

Номера элементов в A-поле и B-поле:

5 – размер COMMON (элемент только для A-поля ?)

6 – внешняя цепочка:
для A-поля: адрес головной цепочки
для B-поля: внешнее имя

7 – точка входа метки:
для A-поля: адрес располож. метки
для B-поля: имя метки

Номера элементов только в A-поле:

8 – External - offset. Used for JMP and CALL to externals.

9 – External + offset. The A value will be added to the two bytes starting at the current location counter immediately before execution.

10 – размер области данных.

11 – счетчик памяти сегмента (ORG cseg/dseg/common).

12 – адрес цепочки. A is head of chain, replace all entries in chain with current location counter. Вход последней цепочки имеет нулевой адрес.

13 – размер области Кода.

14 – конец программного модуля (end program), далее идет выравнивание до границы байта.

Этот элемент не содержит ни A-поле, ни B-поле:

15 – конец файла

Примечание по элементу расширения:

C-поле элемента расширения имеет формат B-поля спец-элемента, но область имени не содержит имя идентификатора, как в B-поле. Вместо него здесь находится один байт типа (сигнатура) элемента расширения, далее следует от 1 до 7 байт дополнительной информации.

Таким образом, каждый элемент расширения имеет формат:

```
1 00b  0100b   zzz  i  jjjjjjjj
               ----------------
                   C-поле
```

zzz любые 3 бита (000b представляется как 8). (001b у асма SOLID).

i 8 бит типа (сигнатура) элемента расширения. (0FEh у асма SOLID).

jjjjjjjj zzz-1 число байт информации. Значение зависит от типа i.

Это присутствует только в одном элементе расширения:

zzz	010b (2)
i	X'35' тип (сигнатура) оверлейного сегмента COBOL
j	номер COBOL-сегмента - 31h (цифровой формат номера)

Когда линкеру встречается сигнатура оверлейного сегмента, текущее число оверлейных сегментов устанавливается в значение j+31h (симв. формат номера).

Если предварительно существующий номер сегмента не был нулевой и включена опция /N (сохр. файл с заданным именем), то область данных записывается на диск в файл. Имя файла равно текущему имени программы, с расширением файла - Vxx, где "xx" две hex-цифры номера сегмента j+31h (симв. формат номера).

Некоторые особенности пакета SOLID C для ОС Estex

СИ-КОМПИЛЯТОР

Коды возврата компилятора (1-й и 2-й проходы):

00h - Ok

0FFh - были errors/warnings, ошибки выделения памяти или работы с файлами.

1-й проход компилятора

1. При компиляции "чужих" исходников ставить опцию "-k".
2. В опциях: -dNAME, -jN, -oNAME и -rP:S:H пробелы не допускаются.
3. Максимальная длина строки полного пути (disk\dir\file.ext) файлов равна 80 символов.
4. Файл "file.err" создается всегда по текущему пути входного .c файла.
5. Формат записи include-файлов только как:

   #include <file.ext> - поиск ведется только в подкаталоге "INCLUDE" расположения компилятора.

   #include "file.ext" - поиск ведется только в каталоге расположения входного .c файла.

Задание пути в имени файла не допускается!. Имя папки "INCLUDE" зарезервировано.

2-й проход компилятора

1. Позволяет компилировать несколько бОльшие исходники, по сравнению с фирменной версией.
2. Исправлены две фирменные ошибки:

   a) нельзя было указывать расширение для TMC-файлов

   b) зависание при компиляции запредельно больших исходников

АССЕМБЛЕР

Примечание: Поскольку ассемблер AS совместим с ассемблером M80 фирмы Microsoft по формату записи ассемблерных текстов - в качестве документации на AS используется документация ассемблера M80. Отличие ассемблера AS от M80 заключается в том, что у AS отсутствуют некоторые команды M80 (в основном, управление вывода листингов).

1. Максимальная длина имен (включающих букву диска и каталоги) файлов равна 64 символа.
2. В псевдокоманде NAME('...') можно использовать также двойные (") кавычки.
3. Имя include-файла можно указывать без расширения. В этом случае будет использоваться расширение по-умолчанию ".asm". Это же правило действует и для входного asm-файла.
4. Поддерживаются три варианта записи недокументированных регистров:

   HX/XH LX/XL

HY/YH LY/YL
HIX/IXH LIX/IXL

5. Псевдокоманды "EQU" и "=" идентичны.

6. Максимальная длина имен глобальных меток, передаваемых в объектный файл (rel-файл) зависит от режима работы ассемблера:
   6 символов - при указании опции "-t"
   8 символов - по-умолчанию
   30 символов - при указании опции "-x" (расширенный rel-формат)
   Следует отметить, что с объектными файлами расширенного rel-формата не работает библиотекарь. Поэтому при компиляции библиотечных файлов, не следует ассемблеру указывать опцию "-x".

7. Коды возврата ассемблера:
   00h - Ok
   0FFh - были errors/warnings, ошибки выделения памяти или работы с файлами.

ЛИНКЕР

При линковке готовых си-программ, собираемых с библиотекой "clib.irl" необходимо всегда указывать линкеру опцию "/gXMAIN". Она передает управление на начальный startup-код си-программ. См. исходники библиотеки "clib".

1. Линкер работает только с именами входных файлов. Указание диска и пути расположения файлов не поддерживается.

2. Исправлены фирменные глюки:

   a) неверная работа с "@list" файлом

   b) не отслеживались запросы на request-файлы

3. При вставке exe-заголовка, параметры равны: ORG = 4100h, Stack = 0C000h. (Значение ORG-а может быть изменено опцией "/r").

4. Файл "list" - содержит одну ком-строку с агрументами. Отдельные аргументы могут начинаться с новой строки. В конце файла должна стоять пустая новая строка. Кол-во читаемых байт ограничено до 256 байт.

5. Поиск библы: сперва файл *.irl, после *.rel. Если не найден, выдается сообщение ошибки: "файл *.rel не найден".

6. Максимальная длина имен идентификаторов - 30 символов.

7. Метки вида ?labels не выводятся в sym-файл и на экран. Становятся "видимыми" только при опции /Q.

8. Отслеживается превышение размера 16kB для dll-библиотек (выдается предупреждение).

9. Коды возврата линкера:
   00h - Ok
   0FFh - были ошибки выделения памяти или работы с файлами.

БИБЛИОТЕКАРЬ

1. Тип открываемого входного файла отслеживается не по его расширению, а по внутреннему содержимому.

2. При создании библиотеки: если имя библиотеки совпадает с именем входного файла, то необходимо указать расширение библиотеки, чтобы не было двойного добавления файлов. В именах rel-файлов расширение можно не указывать. Имена добавляемых файлов разделяются пробелами или запятыми.

3. Глобальные символы "*" и "?" поддерживаются только для режима извлечения модулей (опция "e"). В остальных режимах необходимо указывать реальное имя файла.
4. Задание пути поддерживается только для файла библиотеки. Для rel-файлов указание пути не поддерживается и используется путь расположения библиотеки.
5. В опции l(ist) (список модулей библиотеки), размеры "Code size: ..." и "Data size: ..." выводятся в десятичном формате.
6. В опциях d(ump) и l(ist) вывод на экран можно остановить/продолжить при нажатии на любую клавишу. При нажатии на Esc - выход из режима.
7. Коды возврата библиотекаря:
   00h - Ok
   0FFh - были ошибки работы или операций с файлами.

Оглавление

Компилятор языка Си в системе ESTEX....2
ВВЕДЕНИЕ....2
ГЕНЕРАТОР ОБЪЕКТНОГО КОДА....2
Опции генератора объектного кода....3
КОДОГЕНЕРАТОР....3
Опции кодогенератора....4
ОСОБЕННОСТИ КОМПИЛЯТОРА SOLID C ДЛЯ ОС ESTEX....4
Собственный режим компилятора....4
Режим совместимости со стандартом языка Си....5
Рекурсивные функции....5
Функции с переменными параметрами....6
Преобразования байтовой арифметики....8
Распределение регистров....9
НЕКОТОРЫЕ ДРУГИЕ ОСОБЕННОСТИ КОМПИЛЯТОРА....10
ПЕРЕДАЧА ПАРАМЕТРОВ....14
ВОЗВРАТ ЗНАЧЕНИЯ ФУНКЦИИ....15
ПРАВИЛА ВЫЗОВА ФУНКЦИИ....15
ПЕРЕДАЧИ УПРАВЛЕНИЯ ВЫЗЫВАЕМОЙ ФУНКЦИИ....15
РЕКОМЕНДАЦИИ ПО ПРОГРАММИРОВАНИЮ....17
ЗАРЕЗЕРВИРОВАННЫЕ СЛОВА ЯЗЫКА....17
ДИРЕКТИВЫ КОМПИЛЯТОРА....18
ОПИСАНИЕ ФУНКЦИЙ БИБЛИОТЕКИ СИ....19
ABORT....19
ABS....19
ABSREAD....19
ABSWRITE....20
ATEXIT....20
ATOI....21
BDOS....21
BSEARCH....22
CALLOC....22
CGETS....22
CHDIR....23
CLEARERR....23
CLRSCR....23
CPRINTF....23
CPUTS....24
CREAT....24
DEC8....24
DEC16....24
DEC32....24
DISABLE....25
DIV....25
ENABLE....25
_EXIT....25
EXIT....25
FCLOSE....26
FCLOSALL....26
FDOPEN....26
FFIRST, _FFIRST....27
FFLUSH....27
FGETC....28
FGETPOS....28
FGETS....28
FNEXT....28
FOPEN....29
FPRINTF....29
FPUTC....29
FPUTS....29
FREAD....29
FREE....30
FREOPEN....30
FSCANF....30
FSEEK....31
FSETPOS....31

FTELL....31
FWRITE....31
GETC....32
GETCH....32
GETCHE....32
GETCURSMOUSE....32
GETSENSMOUSE....32
GETGSTATMOUSE....33
GETSTATMOUSE....33
GETCWD....33
GETDATE....34
GETDISK....34
GETENV....34
GETS....34
GETTIME....34
GOTOXY....35
HARDMOUSE....35
HEX8....35
HEX16....35
HEX32....35
HIDEMOUSE....36
HOME....36
INITMOUSE....36
INP....36
INTDOS....36
ISALNUM....37
ISALPHA....37
ISASCII....37
ISATTY....37
ISCNTL....37
ISDIGIT....38
ISGRAPH....38
ISLOWER....38
ISPRINT....38
ISPUNCT....38
ISSPACE....38
ISUPPER....39
ISXDIGIT....39
KBHIT....39
LONGJMP....39
LSEEK....40
LTELL....40
MALLOC....40
MAX....41
MEMCHR....41
MEMCMP....41
MEMCPY....41
MEMMOVE....42
MEMSET....42
MIN....42
MKDIR....42
MOVMEM....42
OPEN....43
OUTP....43
PERROR....43
PRINTF....44
PUTC....44
PUTCH....44
PUTENV....44
PUTS....45
QSORT....45
RAND....45
Описание....46
READ....46
REALLOC....46
REFRESHMOUSE....46
REMOVE....46
RENAME....47

REWIND....47
RMDIR....47
SBRK....48
SCANF....48
SEEK....48
SETDATE....48
SETDISK....49
SETJMP....49
SETCURSMOUSE....49
SETSENSMOUSE....49
SETGPOSMOUSE....50
SETPOSMOUSE....50
SETTXTMOUSE....50
SETMEM....50
SETTIME....50
SHOWMOUSE....51
SLEEP....51
SPRINTF....51
SRAND....51
SSCANF....51
STRCAT....52
STRCHR....52
STRCMP....52
STRCPY....52
STRCSPN....52
STRERR....53
STRLEN....53
STRLWR....53
STRNCAT....53
STRNCMP....54
STRNCPY....54
STRPBRK....54
STRRCHR....54
STRSPN....55
STRSTR....55
STRTOK....55
STRTOL....55
STRUPR....56
SYSENV....56
TELL....57
TOLOWER....57
TOUPPER....57
UNGETC....57
UNLINK....57
VMODEMOUSE....58
WRITE....58
XLIMMOUSE....58
YLIMMOUSE....58
Принцип раздельной компиляции....59
Замечания к исходной программе....59
Процедура компиляции и сборки программы....60
Создание библиотек....61
1. Простой способ создания библиотеки....61
2. Создание полной библиотеки....62
Линкер....66
Создание DLL-библиотек....66
Файл подстановок....67
Библиотекарь....68
IRL формат....68
IRL-заголовок....68
Символьная таблица....69
REL-имидж....69
Формат REL-совместимых объектных файлов....71
Общее представление спец-элемента:....71
Номера элементов контрольного поля....71
Примечание по элементу расширения:....72
Некоторые особенности пакета SOLID C для ОС Estex....74
СИ-КОМПИЛЯТОР....74

1-й проход компилятора.....74
2-й проход компилятора.....74
АССЕМБЛЕР.....74
ЛИНКЕР.....75
БИБЛИОТЕКАРЬ.....75